DeepGroundRepair: From API-Grounded Generation to Shape-Aware Repair for Deep Learning Programs

XIANGYUE MA, Beihang University, China
ZHEN WANG, Beihang University, China
XUHUI LU, Beihang University, China
XIAOTING DU, Beijing University of Posts and Telecommunications, China
CHENGLONG LI, Beijing University of Technology, China
JUN LI, Beihang University, China
HAOTIAN LI, Beihang University, China
ZHENG ZHENG*, Beihang University, China

Deep learning (DL) programs are fundamental to modern AI applications, and automatically generating them from natural-language requirements can substantially reduce DL development effort. Large language models (LLMs) have demonstrated promising capabilities for this direction. However, generated DL programs must satisfy domain-specific API usage constraints and tensor-shape dependencies, violations of which can cause API-level syntactic issues and runtime failures. Existing retrieval-augmented generation and feedback-guided code repair methods do not adequately address these DL-specific challenges.

In this paper, we propose DeepGroundRepair, a three-stage framework that establishes a generation–localization–repair workflow to improve the syntactic correctness and runtime executability of LLM-generated DL programs. Its key idea is to progressively narrow the generation and repair spaces using DL-specific knowledge and runtime tensor information. Specifically, it grounds generation in task-relevant API knowledge, localizes the upstream causes of tensor-shape mismatch failures, and uses runtime tensor shapes to guide parameter repair. Together, these mechanisms transform open-ended generation and repair into a knowledge-grounded and shape-aware process.

We evaluate DeepGroundRepair on the model-level DL programming tasks in DeepEval using six open-weight LLMs across TensorFlow, PyTorch, and PaddlePaddle. We compare it with four baselines: two generation-only methods and their counterparts incorporating traceback-guided self-debugging. The results show that, relative to these baselines, DeepGroundRepair reduces the combined number of API-level syntactic issues by an average of 5.4 per 10 programs and achieves an average relative improvement of 86.0% in code executability. Compared with the two self-debugging baselines, it improves semantic consistency by an average of 16.7% and reduces token cost by an average of 28.4%. The consistent improvements across model families, model scales, and DL libraries provide evidence of its generalizability within the evaluated scope.

*Corresponding author

Authors' Contact Information: Xiangyue Ma, Beihang University, Beijing, China, xiangy_ma@buaa.edu.cn; Zhen Wang, Beihang University, Beijing, China, wz2603520@buaa.edu.cn; Xuhui Lu, Beihang University, Beijing, China, luxuhui@buaa.edu.cn; Xiaoting Du, Beijing University of Posts and Telecommunications, Beijing, China, duxiaoting@bupt.edu.cn; Chenglong Li, Beijing University of Technology, Beijing, China, lichenglong@bjut.edu.cn; Jun Li, Beihang University, Beijing, China, 2503515@buaa.edu.cn; Haotian Li, Beihang University, Beijing, China, lihaotian08@buaa.edu.cn; Zheng Zheng, Beihang University, Beijing, China, zhengz@buaa.edu.cn.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.
© 2026 Copyright held by the owner/author(s). Publication rights licensed to ACM.
Manuscript submitted to ACM

CCS Concepts: • **Software and its engineering → Automatic programming**; *Domain specific languages*; • **Computing methodologies → Artificial intelligence**.

Additional Key Words and Phrases: LLM-based code generation, deep learning programs, retrieval-augmented generation, automated program repair

ACM Reference Format:

Xiangyue Ma, Zhen Wang, Xuhui Lu, Xiaoting Du, Chenglong Li, Jun Li, Haotian Li, and Zheng Zheng. 2026. DeepGroundRepair: From API-Grounded Generation to Shape-Aware Repair for Deep Learning Programs. *ACM Trans. Softw. Eng. Methodol.* XX, X, Article XXX (September 2026), 35 pages. https://doi.org/XXXXXXX.XXXXXXX

1 Introduction

In recent years, large language models (LLMs) have demonstrated strong capabilities in generating code from natural-language requirements [22, 25, 26]. By translating programming intent into executable code, LLMs can make software development more accessible and reduce the effort it requires. However, performance on general-purpose code generation does not necessarily extend to domain-specific programming tasks. This challenge is especially pronounced in DL programming, where structural and semantic dependencies make DL programs prone to bugs involving API usage and tensor compatibility [20, 28, 44]. Therefore, generating syntactically correct and executable DL programs with LLMs remains a significant challenge.

In DL programs, DL models are constructed as computation graphs of interdependent API calls and tensor transformations. Program correctness therefore depends on library-specific API usage and tensor compatibility across the graph. As demonstrated in our prior study [32], LLM-generated DL programs frequently encounter two prevalent problems. ***Problem 1*: API-Level Syntactic Issues.** LLMs frequently hallucinate or misuse DL APIs, including references to nonexistent members and violations of API signatures. ***Problem 2*: Runtime Execution Failures.** LLM-generated DL programs frequently fail at runtime due to violations of implicit constraints among DL operations. Among such failures, tensor-shape mismatches are the predominant type, accounting for 77.34% of execution failures on average [32].

Existing studies relevant to these problems mainly follow two directions. One line of work employs retrieval-augmented generation (RAG) to ground code generation in external knowledge [8, 21, 47]. Its effectiveness, however, depends heavily on retrieving task-relevant context, as irrelevant information can introduce noise and degrade code generation [17]. In DL programming, this challenge is further complicated by the gap between high-level task intent and fine-grained API requirements, making static query–document matching prone to retrieving irrelevant API knowledge [40]. Another line of work investigates feedback-guided code repair, in which failure information from program execution is fed back to LLMs for iterative refinement [11, 38, 46]. However, prior evidence shows that such repair often yields only limited improvements [35]. This limitation is particularly evident in DL programs, where raw tracebacks often reveal only downstream symptoms of tensor-shape errors. Consequently, an LLM reasoning solely from such feedback lacks the evidence needed to localize upstream root causes. Moreover, even after the root cause is localized, self-repair without explicit shape guidance remains insufficient: resolving such errors requires accounting for how multiple interdependent API parameters jointly affect tensor shapes [13].

As discussed above, existing approaches have shown promise in improving general-purpose LLM-generated code, but they do not fully address the two DL-specific problems. Therefore, this work aims to improve the **syntactic correctness** (***Problem 1***) and **runtime executability** (***Problem 2***) of LLM-generated DL programs. To this end, we identify one key retrieval challenge for mitigating API-level syntactic issues during code **generation** and two key challenges concerning the **localization** and **repair** of tensor-shape mismatches underlying runtime execution failures.

- ***Challenge 1 (Generation)*: How to provide LLMs with task-relevant DL API knowledge?** High-level DL task requirements typically describe model structure and functionality rather than explicit API usage, creating a semantic gap between task intent and API documentation.
- ***Challenge 2 (Localization)*: How to localize the root-cause API of a tensor-shape mismatch?** A tensor-shape mismatch may originate from an upstream API configuration but surface only at a downstream operation, separating the failure site from its root cause.
- ***Challenge 3 (Repair)*: How to infer shape-consistent API parameters for repair?** Without explicit guidance on how interdependent API parameters jointly affect tensor shapes, LLMs struggle to identify configurations satisfying tensor-shape constraints.

To address these challenges, we propose DeepGroundRepair, a three-stage framework that establishes a **generation–localization–repair** workflow for DL programs, progressing from API-grounded retrieval-augmented generation through root-cause localization to shape-aware repair. Its main idea is to progressively narrow the generation and repair spaces using DL API knowledge and runtime tensor shapes as complementary guidance. API knowledge grounds code generation, while runtime tensor shapes support root-cause localization and constrain subsequent parameter-level repair. The three stages work in concert to address the challenges outlined above. (1) ***Reasoning-Augmented Classification Retrieval.*** To accurately obtain task-relevant API knowledge, DeepGroundRepair uses LLM-based API recommendation and existence verification to distinguish existing APIs from hallucinated ones, routing existing and nonexistent API candidates to sparse and dense retrieval, respectively. (2) ***Root-Cause API Localization.*** To localize where a tensor shape mismatch should be repaired, DeepGroundRepair combines static program analysis with dynamic execution traces to capture both program structure and runtime behavior. This combined analysis traces the mismatch from its failure site back to the upstream root-cause API and masks suspicious parameters. (3) ***Shape-Aware Self-Repair.*** To determine how the localized API should be repaired, DeepGroundRepair uses traced tensor shapes as explicit evidence and provides the LLM with retrieved shape-related knowledge for the root-cause API. This guidance enables the LLM to infer shape-consistent parameter values while respecting tensor-shape dependencies across the computation graph.

To evaluate the effectiveness of DeepGroundRepair, we conduct experiments on the 100 model-level DL programming tasks in DeepEval using six open-weight LLMs from two model families across TensorFlow, PyTorch, and PaddlePaddle. We compare it against four baselines: two generation-only methods and two methods combining their respective generation strategies with traceback-guided self-debugging. Across the three libraries and four baselines, DeepGroundRepair reduces API hallucinations and parameter misuses by an average of 2.14 and 3.26 issues per 10 programs, respectively. Most notably, it improves code executability by an average of 86.0% relative to the four baselines. Compared with the two self-debugging baselines, DeepGroundRepair achieves a 16.7% average relative improvement in semantic consistency and higher code executability while reducing token cost by 28.4% on average.

In summary, this work makes the following contributions:

- We propose DeepGroundRepair, a novel framework that bridges API-grounded code generation and shape-aware repair, enabling DL programs to be progressively improved through task-relevant API knowledge and runtime tensor-shape feedback.
- We conduct extensive experiments across multiple LLMs and DL libraries. The results demonstrate that DeepGroundRepair generally improves the syntactic correctness, code executability, and semantic consistency of LLM-generated DL programs, while achieving a favorable effectiveness–efficiency trade-off.

- We perform an ablation study and stage-specific analyses to quantify the contributions of the three stages and examine the mechanisms underlying the overall performance gains.
- We release the replication package of DeepGroundRepair [33] to facilitate reproducibility and support future research on LLM-based DL code generation and repair.

2 Motivating Examples

Despite advances in LLM-based code generation, producing syntactically correct and executable DL programs remains challenging, as doing so requires both library-specific API knowledge and effective repair of tensor-shape mismatches.

2.1 Problem 1: API-Level Syntactic Issues.

LLM-generated DL programs frequently contain API-level syntactic issues, including API hallucination and API parameter misuse [32, 42]. Figure 1 presents two representative examples. The first example demonstrates API hallucination, where the generated code invokes the nonexistent `torch.nn.Concatenate`. The second example illustrates API parameter misuse, where `paddle.nn.Conv2D` is configured with the unsupported `channels` parameter while omitting the required `in_channels` and `out_channels` parameters.

[API hallucination]:

```
#[no-member]: Module 'torch.nn' has no 'Concatenate' member.
import torch.nn as nn
...
self.concat = nn.Concatenate(dim=1)
```

[API parameter misuse]:

```
#[unexpected-keyword-arg]: Unexpected keyword argument 'channels' in constructor call.
#[no-value-for-parameter]: No value for argument 'in_channels' in constructor call.
#[no-value-for-parameter]: No value for argument 'out_channels' in constructor call.
import paddle.nn as nn
...
self.conv1 = nn.Conv2D(channels=1, kernel_size=3, padding=1, ...)
```

Fig. 1. Examples of API-level syntactic issues in DL programs generated by the vanilla Gemma-12B model.

(a) DL task requirement (excerpt)	(b) APIs implied by the requirement	(c) Retrieved APIs
... The model will include two sequential feature extraction layers, each consisting of a convolutional layer followed by average pooling. ... After flattening the feature maps, the model will process them through two fully connected layers, each accompanied by a dropout layer to mitigate overfitting. Finally, the model will output classification probabilities via a softmax layer with 1,000 neurons.	.keras.layers.Conv2D .keras.layers.AveragePooling2D .keras.layers.Flatten .keras.layers.Dense .keras.layers.Dropout .keras.layers.Softmax	× .keras.applications.EfficientNetV2M × .image.sobel_edges × .keras.applications.VGG16 ✓ .keras.layers.Softmax × .keras.applications.EfficientNetB7 × .keras.applications.ConvNeXtLarge × .keras.applications.EfficientNetV2B3 × .keras.applications.ResNet50V2 × .keras.applications.EfficientNetB2 × .image.extract_patches

Fig. 2. Example of API retrieval failure, where only one of six required TensorFlow APIs appears among the top 10 results. The `tensorflow.` prefix is omitted from the API names for brevity.

These failures show that generating syntactically correct DL programs requires both identifying APIs that implement the intended operations and using them in accordance with their signatures and parameter specifications.

Existing retrieval methods, however, often fail to provide such knowledge because task requirements express high-level modeling intents, whereas API documentation provides fine-grained, library-specific details. As shown in Figure 2, directly using the complete task requirement as a dense retrieval query fails to bridge this gap. Despite the target architecture implicit in the requirement, most retrieved APIs are irrelevant, and only one of the six required APIs, `tensorflow.keras.layers.Softmax`, appears among the top 10 candidates. This limitation motivates Stage 1, *Reasoning-Augmented Classification Retrieval*, which addresses our first challenge: ***How to provide LLMs with task-relevant DL API knowledge?***

2.2 Problem 2: Runtime Execution Failures

Beyond API-level syntactic issues, LLM-generated DL programs frequently encounter runtime failures. Our prior analysis of 3,677 failures across eight LLMs found tensor-shape mismatches to be the predominant failure type, accounting for 77.34% of execution failures on average [32]. Such mismatches arise from violations of tensor-shape constraints during execution. As illustrated in Figure 3, the generated program fails when the flattened feature tensor is passed to the fully connected layer. Specifically, the flattened tensor contains 401,408 features, whereas the configured `in_features` of `nn.Linear` is 25,088, leading to a shape incompatibility at the layer interface.

[Generated Code]:

```
class DL_Model(nn.Module):
    def __init__(self):
        super(DL_Model, self).__init__()
        ...
        self.avg_pool = nn.AvgPool2d(kernel_size=2, stride=2)
        self.flatten = nn.Flatten()
        self.fc1 = nn.Linear(in_features=512*7*7, out_features=1024)
        ...
    def forward(self, x):
        ...
        x = self.avg_pool(x)
        x = self.flatten(x)
        x = self.fc1(x)
        ...
        return x

model = DL_Model()
input_tensor = torch.randn(1, 3, 224, 224)
output_tensor = model(input_tensor)
```

[Execution failure]:

```
Traceback (most recent call last):
  File "<PROJECT_ROOT>/generated_code.py", line 43, in <module>
    output_tensor = model(input_tensor)
  File "<PROJECT_ROOT>/generated_code.py", line 33, in forward
    x = self.fc1(x)
  ...
  File "<PACKAGES>/torch/nn/modules/linear.py", line 134, in forward
    return F.linear(input, self.weight, self.bias)
RuntimeError:
mat1 and mat2 shapes cannot be multiplied (1x401408 and 25088x1024)
```

Fig. 3. Example of a runtime execution failure in a DL program generated by the vanilla Gemma-12B model.

This failure highlights two key challenges in tensor-shape-aware repair. First, the location where a mismatch is detected does not necessarily correspond to the API configuration that introduces the incompatible shape. In this example, the execution failure is reported at `x=self.fc1(x)`, whereas the faulty parameter configuration is defined earlier when constructing the `nn.Linear` layer in `__init__`. Therefore, execution feedback alone may be insufficient

for root-cause localization, whereas tracing tensor propagation can help identify the API configuration responsible for the mismatch. This observation motivates the design of Stage 2, *Root-Cause Localization*, to address the challenge: ***How to localize the root-cause API of a tensor-shape mismatch?***

Second, after localizing the root-cause API, repairing a mismatch requires inferring parameter values that satisfy tensor-shape constraints rather than performing unconstrained parameter repair. However, runtime tracebacks offer little guidance on how API parameters govern tensor transformations. Moreover, prior studies have shown that LLMs alone struggle to reason about complex numerical constraints among API parameters [13]. Together, these limitations highlight the need for explicit shape guidance in valid parameter inference and motivate the design of Stage 3, *Shape-Aware Self-Repair*, to address the challenge: ***How to infer shape-consistent API parameters for repair?***

The above examples reveal three challenges for LLM-based DL code generation and repair: bridging the task–API knowledge gap, localizing root-cause API configurations, and inferring shape-consistent parameters. Together, these examples also motivate the generation–localization–repair workflow of DeepGroundRepair, which is introduced in the next section.

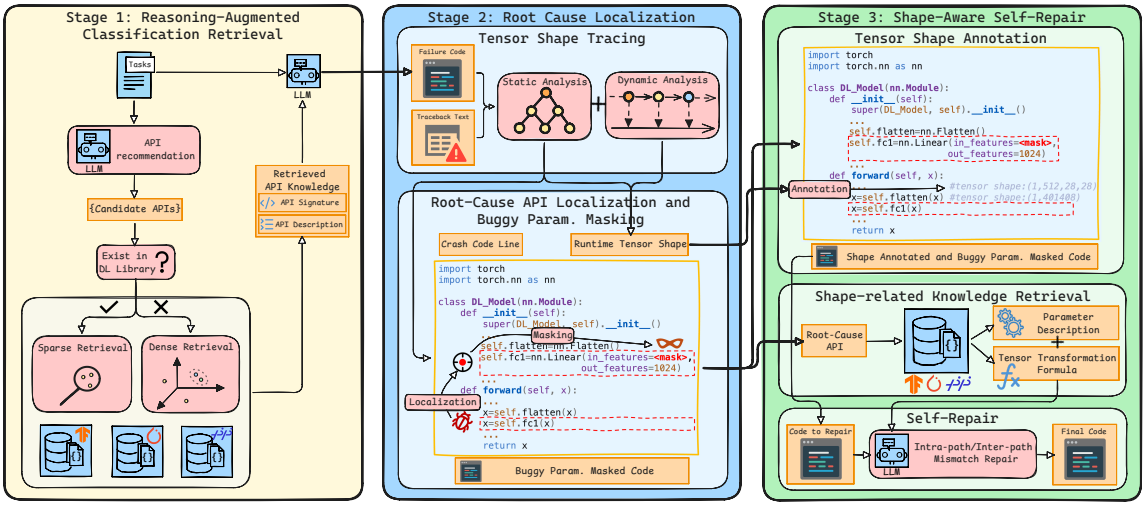


Fig. 4. Overview of DeepGroundRepair, from API-grounded generation through root-cause localization to shape-aware self-repair.

3 Proposed Approach

To address the challenges identified above, we propose DeepGroundRepair, a three-stage framework that implements a generation–localization–repair workflow for DL programs. As illustrated in Figure 4, the framework progresses from *Reasoning-Augmented Classification Retrieval* (Stage 1) for API-grounded code generation, through *Root-Cause Localization* (Stage 2) for narrowing the repair space, to *Shape-Aware Self-Repair* (Stage 3) for deriving shape-consistent repairs. Together, these stages are designed to reduce API-level syntactic issues and improve code executability while preserving semantic consistency.

To support Stages 1 and 3, DeepGroundRepair constructs a library-specific API documentation database shared across the two stages. Rather than providing complete API documentation, the framework selectively retrieves knowledge tailored to each stage: Stage 1 uses API names, signatures, and descriptions to ground draft code generation, whereas

Stage 3 uses the parameter descriptions and tensor-shape transformation formulas of the localized root-cause API to guide repair. Details of database construction are provided in Section 4.4.

The three stages form an integrated pipeline, with the output of each preceding stage facilitating the next. The core idea and design objective of each stage are summarized below.

(1) Stage 1: Reasoning-Augmented Classification Retrieval. This stage aims to reduce factual hallucinations in DL API usage by grounding code generation in task-relevant API knowledge. DeepGroundRepair first recommends the APIs required by the task and verifies their existence. Verified APIs are routed to sparse retrieval, whereas suspected hallucinations are routed to dense retrieval. The retrieved knowledge is then provided to the LLM to generate a more API-compliant draft program. Draft programs with tensor-shape mismatches proceed to Stage 2, whereas the few other execution failures are handled using the traceback-guided strategy adopted by the self-debugging baselines.

(2) Stage 2: Root-Cause Localization. Given a draft program with a tensor-shape mismatch, this stage narrows the repair space by identifying the root-cause API and masking its suspicious parameters. DeepGroundRepair combines abstract syntax tree (AST)-based static analysis with dynamic execution tracing to capture program structure and runtime tensor-shape propagation. The resulting parameter-masked program and traced tensor shapes provide focused context for shape-aware repair in Stage 3.

(3) Stage 3: Shape-Aware Self-Repair. Based on the localization results from Stage 2, this stage applies mismatch-specific shape guidance to repair the buggy program. It first annotates the program with the traced tensor shapes. For intra-path mismatches, the LLM infers the masked parameter values from this local shape context. For inter-path mismatches, DeepGroundRepair additionally retrieves parameter descriptions and, when available, tensor-shape transformation formulas for the root-cause API. It then combines this API knowledge with runtime shape evidence to generate and validate shape-consistent parameter values.

Algorithm 1 Reasoning-Augmented Classification Retrieval

Input: Task requirement R, DL API document database D
Output: Draft DL program P_{draft}

```
candidate_apis = api_recommendation(R)                          /* Step 1: API Recommendation */
for each api in candidate_apis do                               } /* Step 2: API Existence Verification */
    exists = api_existence_check(api)                           }
    if exists then                                              }
        retrieved_docs[api] = sparse_retrieval(api, D)          } /* Step 3: Classification Retrieval */
    else                                                        }
        retrieved_docs[api] = dense_retrieval(api, D)           }
P_draft = LLM(R, retrieved_docs)                                /* Step 4: Generate Draft Program */
return P_draft
```

3.1 Stage 1: Reasoning-Augmented Classification Retrieval

Naive RAG directly retrieves API documentation using high-level task requirements, leaving a semantic gap between DL programming intent and fine-grained API knowledge. Moreover, static query–document matching cannot reliably uncover the APIs implicitly required to implement complex DL model architectures. To bridge this gap, this stage prompts the LLM to reason over the task requirement and identify candidate APIs implicitly required to implement the target DL models. It then verifies these candidates and routes existing and hallucinated candidates to sparse and dense retrieval, respectively. Algorithm 1 summarizes the workflow of this stage, and the individual steps are detailed below.

(1) LLM-Based Reasoning for API Recommendation. To address the semantic gap between task requirements and API documentation, DeepGroundRepair prompts the LLM to reason about which library-specific APIs are implicitly required by each task. The LLM then produces a structured list of candidate API names, translating the task description into an intermediate representation closer to the API documentation space. For reproducibility, we provide the prompt template used for API recommendation in Figure 10.

(2) API Existence Verification. After obtaining the API names recommended by the LLM, DeepGroundRepair verifies each candidate against the target DL library. The probabilistic nature of LLM generation introduces uncertainty into recommendations. Moreover, LLM parametric knowledge of library-specific APIs may be imprecise, particularly for less widely used libraries such as PaddlePaddle. Consequently, the recommendations may include hallucinated API names. Directly using such candidates as queries may return task-irrelevant API documentation, thereby misguiding the LLM during subsequent code generation. To distinguish hallucinated candidates from existing APIs before retrieval, DeepGroundRepair resolves module paths and members using `importlib.import_module` and `getattr`, respectively.

(3) Classification Retrieval. Based on API existence verification, DeepGroundRepair routes each candidate API to either sparse or dense retrieval [31]. This strategy avoids misleading lexical matches caused by hallucinated API names while preserving their semantic information as retrieval clues.

For verified APIs, their canonical names enable exact lexical matching. Sparse retrieval ranks documents based on lexical overlap with a query. Accordingly, DeepGroundRepair uses sparse retrieval to retrieve the API entries from the knowledge base. Given a candidate API name q and an API entry d, their sparse relevance score is computed as:

$$\text{score}_{\text{sparse}}(q,d) = \sum_{t \in q} \text{IDF}(t) \cdot \frac{f(t,d)(k_1+1)}{f(t,d) + k_1 \left(1 - b + b \frac{|d|}{\text{avgdl}}\right)}, \tag{1}$$

where $f(t,d)$ denotes the frequency of term t in d, and k_1 and b are BM25 parameters [37].

For candidate API names determined to be nonexistent, lexical differences from valid API names may obscure relevant matches. Dense retrieval ranks documents based on their proximity to the query in an embedding space. Accordingly, DeepGroundRepair uses dense retrieval to retrieve semantically similar valid APIs. The dense relevance score between q and d is computed as follows:

$$\text{score}_{\text{dense}}(q,d) = -\|E(q) - E(d)\|_2^2, \tag{2}$$

where $E(\cdot)$ denotes the embedding model. Higher scores indicate greater semantic similarity between the candidate API and the retrieved API entry.

(4) LLM-Based Draft Code Generation. After classification retrieval, DeepGroundRepair obtains a set of structured API documents. Rather than providing the full documents to the LLM, Stage 1 retains only the API names, signatures, and descriptions as grounding context for code generation. These fields capture the APIs' core functionality and parameter specifications. The resulting shorter prompt also helps the LLM focus on essential information [30]. Guided by the retrieved API knowledge, the LLM generates a draft DL program following the prompt template shown in Figure 11.

3.2 Stage 2: Root-Cause Localization

Despite API-grounded generation, the draft program may still contain tensor-shape faults that emerge only at runtime. These faults are difficult to diagnose because layer parameters defined in the model initialization method (e.g., `__init__`)

Algorithm 2 Root-Cause Localization

Input: Draft DL program P_{draft}
Output: Masked DL program P_{masked}, root-cause API R, tensor-shape trace T

```
1 obs_points = static_analysis(P_draft)                          } /* Step 1: Tensor Shape Tracing */
2 (T, crash_line) = dynamic_analysis(P_draft, obs_points)        }
3 R = localize_root_api(P_draft, T, crash_line)                  /* Step 2: Root-Cause API Localization */
4 P_masked = mask_buggy_params(P_draft, R, T)                    /* Step 3: Buggy Parameter Masking */
5 return (P_masked, R, T)
```

govern tensor transformations in the forward computation method (e.g., `forward` or `call`)[1], while the resulting mismatches may surface only at downstream operations. Consequently, a traceback typically identifies only the crash site, not the upstream API and parameters causing the mismatch. To address this limitation, Stage 2 backtracks tensor propagation from the crash line to localize the root-cause API and mask its suspicious parameters. This process narrows the repair space and provides focused guidance for Stage 3. Algorithm 2 summarizes the workflow of this stage, and the individual steps are detailed below.

(1) Tensor Shape Tracing. This step reconstructs runtime tensor propagation by tracing intermediate tensor shapes throughout forward execution. DeepGroundRepair first performs AST-based static analysis of the draft program to identify function calls and tensor variables involved in forward computation as candidate observation points. It then executes the program with line-level tracing using `sys.settrace`, recording tensor shapes at these points until an exception occurs. The resulting trace provides explicit tensor-shape context for Stage 3 and auxiliary evidence for root-cause localization.

(2) Root-Cause API Localization. When a crash occurs, DeepGroundRepair locates the failing operation from the traceback and uses AST-based data-flow analysis to trace the involved tensors backward through the forward computation. The recorded shapes reveal how tensor dimensions change along this dependency path, helping identify the layer configuration inconsistent with runtime tensor propagation. The framework then maps the relevant layer instance to its constructor call in `__init__` and identifies the corresponding API as the root cause. The Stage 2 panel of Figure 4 illustrates this process using an excerpt from the tensor-mismatch example in Figure 3. Specifically, the traceback points to `self.fc1(x)` in `forward`, while the shape trace records the dimensions of `x`. The AST analysis then maps `self.fc1` to the `nn.Linear` constructor assigned to it in `__init__`.

(3) Buggy Parameter Masking. Once the root-cause API is localized, DeepGroundRepair selects and masks its potentially responsible parameters. It first maps library-specific layer APIs to unified semantic layer categories, such as convolutional, pooling, and fully connected layers. Each semantic layer category has predefined masking rules for channel and spatial mismatches, specifying which API parameters control the corresponding tensor dimensions. DeepGroundRepair then infers the mismatch type from the traced tensor shapes and applies the corresponding rule for the root-cause API's category. For a channel mismatch, the rule masks parameters governing the expected number of input or output channels, as illustrated by the masked `in_features` argument in the Stage 2 panel of Figure 4. For a spatial mismatch, it masks parameters governing the output height and width, as illustrated by the masked `kernel_size`, `strides`, and `padding` arguments of the root-cause `self.conv_branch` layer in Figure 5.

[1] `forward` is used in PyTorch and PaddlePaddle, whereas `call` is used in TensorFlow.

Algorithm 3 Shape-Aware Self-Repair

Input: Masked DL program P_{masked}, root-cause API R, tensor-shape trace T, DL API document database D
Output: Repaired DL program P_{repaired}

$P_{\text{annotated}}$ = `tensor_shape_annotation`(P_{masked}, T) /* Tensor Shape Annotation */
mismatch = `mismatch_analysis`($P_{\text{annotated}}, T$) /* Mismatch Type Classification */
if *mismatch* = `intra-path` **then**
  P_{repaired} = `LLM_repair`($P_{\text{annotated}}$) } /* Intra-path Mismatch Repair */
else
  K_{shape} = `retrieve_shape_knowledge`(R, D)
  constraints = `build_shape_constraints`(K_{shape}, T)
  Θ = `LLM_generate_parameter_candidates`($P_{\text{annotated}}$, *constraints*)
  Θ' = `constraint_validation`(Θ, *constraints*)
  P_{repaired} = `AST_based_filling`($P_{\text{masked}}, \Theta', R$) } /* Inter-path Mismatch Repair */
return P_{repaired}

3.3 Stage 3: Shape-Aware Self-Repair

Building on the root-cause API and runtime tensor shapes obtained in Stage 2, this stage infers shape-consistent values for the masked parameters. The relevant shape constraints span different dependency scopes. Accordingly, DeepGroundRepair distinguishes between *intra-path mismatches* and *inter-path mismatches* and applies mismatch-specific repair strategies. Intra-path mismatches arise between operations along a single computation path, whereas inter-path mismatches arise when tensors from different paths cannot be merged because of incompatible shapes. Algorithm 3 summarizes the workflow, whose individual steps are described below.

(1) Tensor Shape Annotation. DeepGroundRepair annotates relevant statements in the forward computation with the corresponding tensor shapes traced in Stage 2. For the intra-path mismatch shown in the Stage 3 panel of Figure 4, the tensor passed from the preceding layer to the flattening operation is annotated with shape `(1,512,28,28)`, while the output of `x=self.flatten(x)` is annotated with shape `(1,401408)`. Together, these annotations make explicit that the subsequent `self.fc1` layer must accept 401408 input features. For the inter-path mismatch shown in Figure 5, `branch=self.conv_branch(inputs)` is annotated with its output shape `(1,28,28,16)` and the target shape `(1,14,14,16)` required for merging with the main-path tensor at `Add`. The resulting annotations expose the local or cross-path shape constraints to be satisfied during repair.

(2) Shape-Related API Knowledge Retrieval. For intra-path mismatches, the repair relies on the annotated local shape context without additional retrieval of shape-related API knowledge. For inter-path mismatches, however, the annotations specify the required cross-path transformation but not the parameter values that produce it. Figure 5 illustrates this challenge: the masked parameters of `conv_branch` need to be inferred so that its output matches the main-path tensor before `Add`.

For inter-path mismatch repair, DeepGroundRepair retrieves parameter descriptions and, when available, tensor-shape transformation formulas for the root-cause API to relate its parameters to the target tensor shape. As shown in Figure 6, the retrieved knowledge for `tensorflow.keras.layers.Conv2D` comprises a structured parameter–description mapping and the corresponding transformation formulas. The semantic mapping indicates that `kernel_size`, `strides`, and `padding` govern the output spatial dimensions, whereas `filters` determines the number of output channels. Under the channels-last format[2], the formulas characterize how the tensor shape transforms from $(N, H_{\text{in}}, W_{\text{in}}, C_{\text{in}})$ to

[2] N denotes batch size, while H, W, and C denote height, width, and channel count, respectively. Subscripts in and out distinguish input and output.

```
class DL_Model(tf.keras.Model):
    def __init__(self):
        super(DL_Model, self).__init__()
        self.conv1 = Conv2D(filters=32, kernel_size=(3, 3), strides=(1, 1), padding='same',)
        self.conv2_1 = Conv2D(filters=16, kernel_size=(1, 1), strides=(1, 1), padding='same')
        self.conv2_2 = Conv2D(filters=16, kernel_size=(1, 1), strides=(1, 1), padding='same')
        self.maxpool = MaxPool2D(pool_size=(2, 2), strides=(2, 2), padding='same')
-       self.conv_branch = Conv2D(filters=16, kernel_size=(3, 3), strides=(1, 1), padding='same')
+       self.conv_branch = Conv2D(filters=16, kernel_size=<mask>, strides=<mask>, padding=<mask>)
        self.add = Add()
        ...
    def call(self, inputs):
        x = self.conv1(inputs)                                  # output tensor shape: (1,28,28,32)
        x = self.conv2_1(x)                                     # output tensor shape: (1,28,28,16)
        x = self.conv2_2(x)                                     # output tensor shape: (1,28,28,16)
        x = self.maxpool(x)                                     # output tensor shape: (1,14,14,16)
        x = self.dropout(x)                                     # output tensor shape: (1,14,14,16)
        branch = self.conv_branch(inputs)                       # output tensor shape: (1,28,28,16)
                                                                # target tensor shape: (1,14,14,16)
        merged = self.add([x, branch])
        ...
        return output

model = DL_Model()
input_tensor = tf.random.normal((1, 28, 28, 1))
output_tensor = model(input_tensor)
```

Fig. 5. Example of an inter-path mismatch in a program generated by Gemma-12B. Stage 2 masks the potentially buggy parameters during root-cause API initialization in `__init__`, while Stage 3 annotates tensor shapes along the computation paths in `call`.

Shape-related Knowledge: `tensorflow.keras.layers.Conv2D`

API Parameter Semantics	
`filters`	Specifies the number of output filters.
`kernel_size`	Defines the height and width of the convolution window.
`strides`	Determines the step size of the convolution operation.
`padding`	Specifies whether valid or same padding is applied.

Tensor Shape-Transformation Formulas	
C (Channel)	C_{out} = filters
H (Height)	$H_{out} = \lfloor (H_{in} + 2 \cdot \text{padding}_h - \text{kernel_size}_h) \,/\, \text{stride}_h + 1 \rfloor$
W (Width)	$W_{out} = \lfloor (W_{in} + 2 \cdot \text{padding}_w - \text{kernel_size}_w) \,/\, \text{stride}_w + 1 \rfloor$

Fig. 6. Example of shape-related knowledge of `tensorflow.keras.layers.Conv2D`.

$(N, H_{\text{out}}, W_{\text{out}}, C_{\text{out}})$ as governed by the corresponding API parameters. Together, the parameter descriptions and tensor shape transformation formulas guide the LLM in coordinating parameter values to satisfy cross-path shape constraints.

(3) Shape-Aware Code Repair. To account for their different dependency scopes, DeepGroundRepair adopts distinct repair strategies for intra-path and inter-path mismatches. For intra-path mismatches, DeepGroundRepair prompts the LLM to generate a repaired code line from the shape-annotated and parameter-masked program. The generated line then replaces the corresponding faulty API call in the draft program.

For inter-path mismatches, DeepGroundRepair adopts a generate-and-validate strategy. In the example shown in Figure 5, the shape of the main-path tensor entering `Add` defines the target shape for the output of the shortcut-path `Conv2D`. Using annotations that capture this target shape and tensor-shape propagation process, together with retrieved shape-related knowledge for `Conv2D`, the LLM proposes candidate combinations for the masked parameter values.

A lightweight constraint solver then evaluates each combination by checking whether the resulting `Conv2D` output matches the target shape. Finally, the validated combination, comprising `kernel_size=(3,3)`, `strides=(2,2)`, and `padding="same"`, is inserted into the masked `Conv2D` call to produce the repaired program. For reproducibility, the prompt templates for intra-path and inter-path repair are provided in Figures 12 and 13, respectively.

4 Experimental Setup

To systematically evaluate the proposed DeepGroundRepair framework, we organize our experimental study around the following five research questions (RQs). We also describe the baselines, evaluation metrics, benchmark, and implementation details used in our evaluation.

- **RQ1: End-to-End Effectiveness.** How does DeepGroundRepair perform end to end compared with the baselines?
- **RQ2: Ablation Study.** How does removing each key component affect the end-to-end performance of DeepGroundRepair?
- **RQ3: Stage-specific Analysis.** How effectively do the three stages of DeepGroundRepair address the corresponding challenges in generation, localization, and repair for DL programs?
- **RQ4: Generalizability.** How well do the performance gains of DeepGroundRepair generalize across different model families, model scales, and DL libraries?
- **RQ5: Token Efficiency.** How does DeepGroundRepair balance effectiveness and token cost compared with the baselines?

In brief, we evaluate the **effectiveness** of DeepGroundRepair in RQ1, assess individual component **contributions** through an ablation study in RQ2, analyze the underlying **mechanisms** via stage-specific analysis in RQ3, investigate the **generalizability** of our approach in RQ4, and assess its **efficiency** in RQ5. For evaluation, we employ end-to-end effectiveness metrics for RQ1, RQ2, and RQ4, while stage-specific effectiveness metrics are used for RQ3. The detailed definitions of all metrics are provided in Section 4.2.

4.1 Baselines

To evaluate the effectiveness of DeepGroundRepair, we compare it with four baselines: two generation-only baselines (*Baselines 1 and 2*) and their self-debugging counterparts (*Baselines 3 and 4*).

4.1.1 Baseline 1: Vanilla LLM. This baseline uses zero-shot prompting to generate DL programs directly from the task requirements. It evaluates the model's fundamental ability to generate DL programs.

4.1.2 Baseline 2: Hybrid-RAG. Following prior work [16], we adapt Hybrid-RAG as a baseline that combines sparse and dense retrievers to capture both lexical and semantic similarity. The two retrievers independently produce ranked lists of API documents. We fuse these lists using reciprocal rank fusion (RRF) [12], which assigns each API document d appearing in either list the following score:

$$\mathrm{RRF}(d) = \sum_{j=1}^{m} \frac{w_j}{c + r_j(d)}, \tag{3}$$

where w_j is the weight of the j-th retriever, $r_j(d)$ is the rank of document d in its result list, and $c = 60$ is the rank constant. Following standard RRF, we assign equal weights to both retrievers ($w_j = 1$) and provide the top-10 fused documents to the LLM for code generation.

4.1.3 Baseline 3: Vanilla LLM with Self-Debugging. Prior work shows that execution feedback helps LLMs diagnose and repair errors in generated programs [11, 38]. Following this feedback-guided paradigm, *Baseline 3* augments vanilla LLM generation with traceback-guided self-debugging. It generates a draft program using the same setting as *Baseline 1* and executes it. If execution fails, the LLM performs a repair iteration using the resulting runtime traceback as feedback.

4.1.4 Baseline 4: Hybrid-RAG with Self-Debugging. To evaluate the combined effectiveness of retrieval-augmented generation and feedback-guided repair, *Baseline 4* uses Hybrid-RAG to generate a draft program and then executes it. If execution fails, the LLM performs a repair iteration using the resulting runtime traceback as feedback.

4.2 Evaluation Metrics

This section presents the evaluation metrics used to compare DeepGroundRepair with baseline methods. The metrics are categorized into three groups according to their evaluation objectives: end-to-end effectiveness metrics for RQ1, RQ2, and RQ4; stage-specific effectiveness metrics for RQ3; and the token efficiency metric for RQ5.

4.2.1 End-to-End Effectiveness Metrics. To evaluate the overall effectiveness of DeepGroundRepair compared to baseline methods, we assess the final DL programs from three complementary perspectives: static checking, dynamic checking, and semantic checking.

(1) Static Checking. We employ Pylint [6], a widely used static analysis tool, to identify API-level syntactic issues. We focus on the *Error* messages reported by Pylint[3], which indicate severe issues that may affect program correctness. Based on their categories, we define two metrics: *API Hallucination Count* and *API Parameter Misuse Count*. Both metrics are computed by counting the issues reported by Pylint in their corresponding categories.

• ***API Hallucination Count*** quantifies references to nonexistent APIs, including `no-member`, `import-error`, and `no-name-in-module`.

• ***API Parameter Misuse Count*** quantifies violations of API signatures, including `unexpected-keyword-arg`, `no-value-for-parameter`, `too-many-function-args`, and `redundant-keyword-arg`.

(2) Dynamic Checking. We execute each generated DL program and measure its *Code Executability Rate* (*CER*).

• ***Code Executability Rate*** measures the proportion of generated programs that can be executed successfully without runtime errors. It reflects whether these programs satisfy the basic usability requirements for DL tasks. Let $\mathcal{D}$ denote the set of generated programs and $\mathcal{D}_{\text{exec-success}}$ denote the subset of executable programs. *CER* is defined as:

$$CER = \frac{|\mathcal{D}_{\text{exec-success}}|}{|\mathcal{D}|}. \tag{4}$$

(3) Semantic Checking. We introduce *Semantic Consistency Rate* (*SCR*) to evaluate whether repaired programs preserve the semantics of their draft versions.

Semantic Consistency Rate provides a practical approximation of semantic preservation based on effective-line-count preservation, because directly determining semantic equivalence requires labor-intensive, fine-grained analysis of program behavior and structure. During self-repair, an LLM may make a program executable by altering its DL model structure, such as adding or removing layers, rather than resolving the failure while preserving the existing architecture.

[3]We exclude other Pylint message categories (*Fatal*, *Refactor*, *Warning*, and *Convention*) because they are rare or capture lower-severity issues. We also filter out false positives due to TensorFlow's lazy-loading mechanism, such as the `import-error` report for `tensorflow.keras.layers`.

Therefore, an unchanged effective-line count indicates that the repair is less likely to introduce such coarse-grained structural changes, although this proxy does not establish semantic equivalence. Specifically, *SCR* is defined as:

$$SCR = \frac{|\mathcal{D}_{\text{preserved}}|}{|\mathcal{D}|}. \tag{5}$$

4.2.2 Stage-specific Effectiveness Metrics. To evaluate whether each stage achieves its intended objective, we employ dedicated metrics tailored to the corresponding stage of DeepGroundRepair.

(1) API Knowledge Retrieval. We evaluate Stage 1 by comparing the set of top-K retrieved APIs with the ground-truth API set annotated for each task. Based on their overlap, we compute *Precision@K* and *Recall@K*. Let G denote the ground-truth API set and R^K the set of top-K retrieved APIs.

• ***Precision@K*** measures the proportion of relevant APIs among the top-K retrieved APIs. The definition is as follows.

$$Precision@K = \frac{|R^K \cap G|}{K}. \tag{6}$$

• ***Recall@K*** measures the proportion of ground-truth APIs covered by the top-K retrieved APIs. The definition is as follows.

$$Recall@K = \frac{|R^K \cap G|}{|G|}. \tag{7}$$

(2) Root-Cause Identification. We evaluate Stage 2 using *Root-Cause API Localization Accuracy*.

• ***Root-Cause API Localization Accuracy*** measures the proportion of initial shape-mismatch failures for which Stage 2 correctly identifies the root cause API that is responsible for the mismatch. Let $\mathcal{D}_{\text{shape-error}}$ denote the set of programs with initial shape-mismatch failures, and let $\mathcal{D}_{\text{root-localized}}$ denote the subset for which the localized root-cause API matches the manually annotated root-cause API. We define *RCLA* as:

$$RCLA = \frac{|\mathcal{D}_{\text{root-localized}}|}{|\mathcal{D}_{\text{shape-error}}|}. \tag{8}$$

(3) Shape-Aware Repair. We evaluate Stage 3 using *Shape-Mismatch Error Repair Success Rate* (*ShapeRSR*).

• ***Shape-Mismatch Error Repair Success Rate***. Following prior work [38], we adapt *Repair Success Rate* to measure successful fixes of shape-mismatch errors. Let $\mathcal{D}_{\text{shape-error}}$ denote the programs that initially fail due to shape mismatches, and $\mathcal{D}^{\text{repaired}}_{\text{shape-error}}$ denote the successfully repaired subset. *ShapeRSR* is defined as:

$$ShapeRSR = \frac{|\mathcal{D}^{\text{repaired}}_{\text{shape-error}}|}{|\mathcal{D}_{\text{shape-error}}|}. \tag{9}$$

4.2.3 Token Efficiency. We examine token cost alongside effectiveness to evaluate token efficiency of each method. For each model–library configuration, token cost is measured as the average number of completion tokens across 100 tasks.

For generation-only baselines (*Baseline 1* and *Baseline 2*):

$$Token\ Cost = T_{\text{generation}}. \tag{10}$$

For methods with a repair stage (*Baseline 3*, *Baseline 4*, and DeepGroundRepair)[4]:

$$Token\ Cost = T_{\text{generation}} + T_{\text{repair}}. \tag{11}$$

[4]For *Baselines 3 and 4*, $T_{\text{generation}}$ is taken from *Baselines 1 and 2*, respectively; for DeepGroundRepair, it includes all completion tokens generated in Stage 1. If the draft program executes successfully, no repair is performed and $T_{\text{repair}} = 0$.

4.3 Benchmark

In this work, we adopt DeepEval [32] as our evaluation benchmark. To the best of our knowledge, DeepEval was the only benchmark specifically designed to evaluate model-level DL code generation when this study was conducted. It comprises 100 tasks, each consisting of a natural-language requirement and a reference implementation. The requirement specifies the model inputs, architecture, and outputs. Unlike general-purpose code generation benchmarks centered on standalone functions [10, 39], DeepEval focuses on the construction of complete DL models, enabling the evaluation of LLM-generated DL programs at the model level.

We evaluate all methods on the complete DeepEval benchmark. Its diversity in model architectures and task complexity provides a broad and consistent basis for comparing different methods. Moreover, the library-independent task descriptions allow the same tasks to be evaluated consistently across TensorFlow, PyTorch, and PaddlePaddle. For the retrieval evaluation in Section 5.3.1, we derive library-specific ground-truth API sets by mapping APIs in the Keras reference implementations to their counterparts in TensorFlow, PyTorch, and PaddlePaddle.

4.4 Implementation

4.4.1 API Document Database. To support retrieval in Stages 1 and 3 of DeepGroundRepair, we construct library-specific API documentation databases for the three target DL libraries. We first use *BeautifulSoup* to parse the official online API documentation of TensorFlow[5], PyTorch[6], and PaddlePaddle[7], collecting 3,686, 4,137, and 1,531 API documents, respectively. We then preprocess the raw documents using GPT-5.4-mini to remove noise and shorten subsequent prompts, thereby helping the LLM focus on essential API information [30]. Specifically, we normalize API signatures, remove nonessential parameters, and condense API and parameter descriptions. Finally, we organize the processed documents into library-specific databases and build separate sparse and dense retrieval indexes for each database.

4.4.2 Retrieval Configuration. Using the constructed API documentation databases, we configure both sparse and dense retrieval. For sparse retrieval, we use the BM25 algorithm [37], implemented with the `Whoosh` package [7], to rank API documents based on lexical matches between each query and the structured API fields. For dense retrieval, we use the BGE-M3 model [9] to encode queries and documents into embedding vectors. FAISS [15] indexes the document vectors and performs similarity search. In DeepGroundRepair, existing and nonexistent API candidates are routed to sparse and dense retrieval, respectively. For each API candidate, we retain the highest-ranked retrieved document, yielding up to 10 API documents as context for the LLM. By contrast, Hybrid-RAG applies both retrieval methods to each query and fuses their ranked results using reciprocal rank fusion (RRF) [12], retaining the top 10 documents after fusion.

4.4.3 Studied LLMs. As detailed in Table 1, we evaluate LLMs from the Gemma and Mistral families, covering three parameter-scale ranges within each family: below 10B, 10B–20B, and above 20B. This selection enables us to examine the generalizability of DeepGroundRepair across model families and scales, as presented in Section 5.4. All selected models are open-weight and publicly available on HuggingFace [5], allowing local deployment while supporting transparent, reproducible evaluation.

4.4.4 Environment. To ensure a fair comparison, we evaluate all methods using identical hardware and software configurations. Specifically, all experiments are conducted on a platform equipped with two NVIDIA V100 GPUs and

[5] https://www.tensorflow.org/api_docs/python/tf
[6] https://docs.pytorch.org/docs/2.11/pytorch-api.html
[7] https://www.paddlepaddle.org.cn/documentation/docs/en/api/index_en.html

Table 1. Studied LLMs.

Model	Abbr.	Release Time	Size	Team	Open Weight?
Gemma-3-4B-it [3]	Gemma-4B	2025.03	4B	Google	✓
Gemma-3-12B-it [3]	Gemma-12B	2025.03	12B	Google	✓
Gemma-3-27B-it [3]	Gemma-27B	2025.03	27B	Google	✓
Ministral-3-3B-Instruct [29]	Mistral-3B	2026.01	3B	Mistral AI	✓
Ministral-3-14B-Instruct [29]	Mistral-14B	2026.01	14B	Mistral AI	✓
Mistral-Small-3.2-24B-Instruct [4]	Mistral-24B	2025.06	24B	Mistral AI	✓

running CentOS Linux 7.8.2003 (Core), Python 3.10.0, CUDA 12.1, and cuDNN 8.9.5.29. Within this environment, we use TensorFlow 2.16.1, PyTorch 2.11.0, and PaddlePaddle 3.3.0, which were the latest stable releases available when the experiments were conducted. For decoding, we adopt nucleus sampling [18] with top-p set to 0.95 for all models. We set the temperature to 0.8 for Gemma models and 0.1 for Mistral models; the latter follows the decoding guidance in the official Mistral model cards.[8] Finally, to ensure sufficient capacity for complete outputs, we set the maximum generation length to 2,000 tokens for draft code generation and 3,000 tokens for code repair.

5 Experimental Results

5.1 RQ1: End-to-End Effectiveness

To assess the end-to-end effectiveness of DeepGroundRepair, we evaluate whether it alleviates the two problems identified in Section 1: API-level syntactic issues and runtime execution failures. We present the detailed results in the order of static checking, dynamic checking, and semantic checking, using the metrics defined in Section 4.2.1.

5.1.1 Static checking. We evaluate two API-level syntactic issues: API hallucination and API parameter misuse. As shown in Table 2, when averaged across the three libraries and four baselines, DeepGroundRepair produces 2.14 fewer API hallucinations and 3.26 fewer API parameter misuses per 10 programs.

For API hallucination, DeepGroundRepair achieves the lowest average *API Hallucination Count* on TensorFlow and PaddlePaddle, with 0.10 and 3.33 issues per 10 programs, respectively. Compared with the four baselines, it reduces hallucinations by 0.25–0.71 issues on TensorFlow and 4.09–7.10 issues on PaddlePaddle. The particularly large reductions on PaddlePaddle suggest that DeepGroundRepair is especially beneficial when LLMs have limited parametric knowledge of less widely used DL libraries. On PyTorch, DeepGroundRepair outperforms *Baselines 1–3* and remains close to *Baseline 4*, with only 0.18 more issues per 10 programs.

For API parameter misuse, DeepGroundRepair achieves the lowest average *API Parameter Misuse Count* on TensorFlow, reducing the count to 0.84 and producing 2.79–4.51 fewer issues than the baselines. On PyTorch, all methods exhibit near-zero misuse counts, leaving little room for improvement. On PaddlePaddle, DeepGroundRepair substantially outperforms *Baseline 1* and *Baseline 3*, remains comparable to *Baseline 2*, and produces 0.29 more issues than *Baseline 4*. Thus, the reductions are most pronounced in settings where the baselines exhibit substantial API misuse.

[8] https://huggingface.co/mistralai/Ministral-3-3B-Instruct-2512.

Table 2. *API Hallucination Count* and *API Parameter Misuse Count* per 10 programs across models and DL libraries. DeepGR denotes DeepGroundRepair, and B1–B4 denote Baselines 1–4, respectively. "Change" denotes the performance difference between DeepGR and each baseline, calculated based on the values in the "Overall Avg." row. The "↓" and "↑" arrows indicate fewer and more issues, respectively, and bold values indicate the best result in each row.

Model	TensorFlow					PyTorch					PaddlePaddle				
	B1	B2	B3	B4	DeepGR	B1	B2	B3	B4	DeepGR	B1	B2	B3	B4	DeepGR
API Hallucination Count															
Gemma-4B	0.00	3.84	0.00	2.63	**0.00**	9.50	4.50	8.30	4.10	**3.74**	34.8	22.7	30.9	21.7	**11.7**
Gemma-12B	1.60	0.40	1.60	0.40	**0.40**	3.80	1.30	1.30	1.10	1.60	3.90	8.70	4.30	6.70	4.29
Gemma-27B	0.60	0.50	0.40	0.50	**0.00**	3.70	1.40	0.30	0.30	0.80	9.80	8.40	6.70	6.00	**1.40**
Average	0.73	1.58	0.67	1.18	**0.13**	5.67	2.40	3.30	1.83	2.05	16.2	13.3	14.0	11.5	**5.80**
Mistral-3B	0.41	0.00	0.00	0.00	**0.00**	0.90	0.80	0.80	0.80	0.84	13.1	8.60	8.60	8.60	**2.47**
Mistral-14B	0.20	0.10	0.10	0.20	**0.00**	0.20	0.00	0.00	0.00	0.30	0.60	0.40	0.40	0.40	**0.00**
Mistral-24B	0.00	0.00	0.00	0.00	0.20	0.31	0.00	0.00	0.00	0.10	0.40	1.10	1.10	1.10	**0.10**
Average	0.20	0.03	0.03	0.07	0.07	0.47	0.27	0.27	0.27	0.41	4.70	3.37	3.37	3.37	**0.86**
Overall Avg.	0.47	0.81	0.35	0.62	**0.10**	3.07	1.33	1.78	1.05	1.23	10.43	8.32	8.67	7.42	**3.33**
Change	**↓0.37**	**↓0.71**	**↓0.25**	**↓0.52**	–	**↓1.84**	**↓0.10**	**↓0.55**	↑0.18	–	**↓7.10**	**↓4.99**	**↓5.34**	**↓4.09**	–
API Parameter Misuse Count															
Gemma-4B	3.50	0.91	3.90	0.91	**0.41**	0.00	0.00	0.00	0.00	**0.00**	76.8	3.94	65.6	4.04	11.2
Gemma-12B	3.80	3.20	4.00	3.20	**1.51**	0.00	0.00	0.30	0.00	0.30	14.3	4.00	10.2	2.10	2.86
Gemma-27B	6.80	1.00	5.60	1.20	**0.20**	0.00	0.00	0.00	0.00	0.10	0.10	2.00	0.00	1.90	0.70
Average	4.70	1.70	4.50	1.77	**0.71**	0.00	0.00	0.10	0.00	0.13	30.4	3.31	25.27	2.68	4.92
Mistral-3B	5.98	4.60	4.69	4.60	**1.58**	0.00	0.00	0.00	0.00	**0.00**	9.20	5.80	5.80	5.80	**1.13**
Mistral-14B	5.00	5.05	5.00	5.05	**0.71**	0.20	0.20	0.20	0.20	**0.00**	0.80	0.80	0.80	0.80	**0.41**
Mistral-24B	7.00	7.00	7.00	7.00	**0.61**	0.20	0.00	0.00	0.00	**0.00**	0.30	0.10	0.10	0.10	0.20
Average	5.99	5.55	5.56	5.55	**0.97**	0.13	0.07	0.07	0.07	**0.00**	3.43	2.23	2.23	2.23	**0.58**
Overall Avg.	5.35	3.63	5.03	3.66	**0.84**	0.07	0.03	0.08	0.03	0.07	16.92	2.77	13.75	2.46	2.75
Change	**↓4.51**	**↓2.79**	**↓4.19**	**↓2.82**	–	0.00	↑0.04	**↓0.01**	↑0.04	–	**↓14.17**	**↓0.02**	**↓11.00**	↑0.29	–

5.1.2 Dynamic checking. Static checking captures API-level syntactic issues but cannot reveal runtime failures. We therefore execute the final programs and measure their executability using *CER*. As shown in Table 3, when averaged across the three libraries and four baselines, DeepGroundRepair achieves a relative *CER* improvement of 86.0%.

In particular, DeepGroundRepair achieves the highest overall *CER* on all three DL libraries, reaching 58.2% on TensorFlow, 41.2% on PyTorch, and 31.5% on PaddlePaddle. Compared with the strongest baseline for each library, DeepGroundRepair yields relative gains of 13.7%, 31.6%, and 61.5%, respectively. Compared with the generation-only baselines, it improves *CER* by 54.0%–279.5%, showing the benefit of targeted repair for failed draft programs. It also outperforms the self-debugging baselines by 13.7%–77.0%, demonstrating that the complete DeepGroundRepair pipeline is more effective than existing traceback-guided self-debugging methods.

Across 18 model–library configurations, DeepGroundRepair achieves the highest *CER* in 15 configurations and ties for the highest in one. The only two configurations in which it is outperformed are Mistral-14B and Mistral-24B on TensorFlow, where *Baseline 4* achieves *CER* of 66% and 83%, compared with 59% and 66% for DeepGroundRepair. However, our results show that these gains in executability are accompanied by lower semantic consistency, underscoring the need to consider both metrics jointly, as discussed below.

Table 3. *Code Executability Rate* (*CER*, %) of each model across DL libraries and methods. DeepGR denotes DeepGroundRepair, and B1–B4 denote Baselines 1–4, respectively. "Δ (%)" reports the relative change of DeepGR over each corresponding baseline using the values in the "Overall Avg." row. The "↑" arrow indicates improvements, and bold values indicate the best result in each row.

Model	TensorFlow					PyTorch					PaddlePaddle				
	B1	B2	B3	B4	DeepGR	B1	B2	B3	B4	DeepGR	B1	B2	B3	B4	DeepGR
Gemma-4B	36	20	38	20	**46**	6	9	6	9	**25**	0	0	0	6	**12**
Gemma-12B	36	26	43	32	**59**	18	19	21	24	**34**	2	2	5	5	**21**
Gemma-27B	44	38	55	49	**74**	19	28	28	34	**39**	11	5	21	12	**36**
Average	38.7	28.0	45.3	33.7	**59.7**	14.3	18.7	18.3	22.3	**32.7**	4.3	2.3	8.7	7.7	**23.0**
Mistral-3B	13	29	33	44	**45**	3	15	8	21	**38**	1	4	6	12	**20**
Mistral-14B	51	47	63	66	59	31	32	49	43	**54**	27	14	35	29	**48**
Mistral-24B	47	60	75	83	66	25	33	47	57	**57**	32	25	50	43	**52**
Average	37.0	45.3	**57.0**	64.3	56.7	19.7	26.7	34.7	40.3	**49.7**	20.0	14.3	30.3	28.0	**40.0**
Overall Avg.	37.8	36.7	51.2	49.0	**58.2**	17.0	22.7	26.5	31.3	**41.2**	12.2	8.3	19.5	17.8	**31.5**
Δ (%)	**↑54.0**	**↑58.6**	**↑13.7**	**↑18.8**	–	**↑142.4**	**↑81.5**	**↑55.5**	**↑31.6**	–	**↑158.2**	**↑279.5**	**↑61.5**	**↑77.0**	–

Table 4. *Semantic Consistency Rate* (*SCR*, %) of each model across DL libraries and methods. DeepGR denotes DeepGroundRepair, and B3–B4 denote Baselines 3–4, respectively. "Δ (%)" reports the relative change of DeepGR over each corresponding baseline using the values in the "Overall Avg." row. The "↑" arrow indicates improvements, and bold values indicate the best result in each row.

Model	TensorFlow			PyTorch			PaddlePaddle		
	B3	B4	DeepGR	B3	B4	DeepGR	B3	B4	DeepGR
Gemma-4B	**97**	89	94	76	79	**94**	70	91	**95**
Gemma-12B	86	89	**92**	91	90	**99**	71	**93**	86
Gemma-27B	86	91	**97**	86	91	**97**	83	89	**100**
Average	89.7	89.7	**94.3**	84.3	86.7	**96.7**	74.7	91.0	**93.7**
Mistral-3B	60	61	**84**	72	56	**91**	73	47	**92**
Mistral-14B	74	75	**97**	83	87	**94**	81	79	**90**
Mistral-24B	76	83	**91**	78	88	**96**	78	83	**91**
Average	70.0	73.0	**90.7**	77.7	77.0	**93.7**	77.3	69.7	**91.0**
Overall Avg.	79.8	81.3	**92.5**	81.0	81.8	**95.2**	76.0	80.3	**92.3**
Δ (%)	**↑15.9**	**↑13.8**	–	**↑17.5**	**↑16.4**	–	**↑21.4**	**↑14.9**	–

5.1.3 Semantic checking. To assess whether improved executability comes at the cost of reduced semantic consistency, we use *SCR* to compare each draft program with its final version. As shown in Table 4, DeepGroundRepair achieves average *SCR* values of 92.5%, 95.2%, and 92.3% on TensorFlow, PyTorch, and PaddlePaddle, respectively. At the configuration level, it obtains the highest *SCR* in 16 of the 18 model–library configurations. Overall, these results represent an average relative improvement of 16.7% over the two self-debugging baselines across the three libraries.

The importance of jointly considering *CER* and *SCR* is particularly evident for Mistral-14B and Mistral-24B on TensorFlow. These are the only two configurations in which *Baseline 4* achieves a higher *CER* than DeepGroundRepair. However, its corresponding *SCR* values are only 75% and 83%, falling 22 and 8 percentage points below DeepGroundRepair's 97% and 91%, respectively. This contrast shows that the *CER* advantage of *Baseline 4* in these configurations is accompanied by substantially lower semantic consistency.

Summary of Answers to RQ1

Overall, DeepGroundRepair demonstrates strong end-to-end effectiveness across all three evaluation dimensions. Averaged across the evaluated comparisons, it produces 2.14 fewer API hallucinations and 3.26 fewer API parameter misuses per 10 programs and achieves an average relative *CER* improvement of 86.0%. Compared with the self-debugging baselines, it also improves *SCR* by an average of 16.7%. These results show that DeepGroundRepair substantially improves runtime executability while generally reducing API-level syntactic issues and achieving higher semantic consistency between draft and repaired programs.

5.2 RQ2: Ablation Study

To quantify each component's contribution, we conduct an ablation study by removing one stage at a time. To balance model-family coverage and experimental cost, we use the medium-sized model from each family, Gemma-12B and Mistral-14B. We evaluate each variant through static, dynamic, and semantic checking, following the procedure and end-to-end effectiveness metrics described in Sections 5.1 and 4.2.1, respectively. The variants are described below.

- *Variant 1* (w/o *Reasoning-Augmented Classification Retrieval*): Removes retrieved DL API knowledge and instead uses vanilla LLM generation to produce draft programs.
- *Variant 2* (w/o *Root-Cause Localization*): Removes the localized root-cause API and parameter masks from the repair context, using only the raw runtime traceback as repair guidance.
- *Variant 3* (w/o *Shape-Aware Self-Repair*): Removes tensor-shape annotations and shape-related API knowledge from the repair context, leaving the LLM without explicit shape guidance to repair the programs.

Table 5. *API Hallucination Count* and *API Parameter Misuse Count* per 10 programs for the complete DeepGroundRepair pipeline and its three ablated variants, denoted in the table by DeepGR and V1–V3, respectively. "*Average*" denotes the arithmetic mean across the two models. Superscript values denote the performance difference between each variant and DeepGR, calculated from the "*Average*" values. The "↓" and "↑" symbols indicate lower and higher error counts, respectively.

Model	TensorFlow				PyTorch				PaddlePaddle			
	DeepGR	V1	V2	V3	DeepGR	V1	V2	V3	DeepGR	V1	V2	V3
API Hallucination Count												
Gemma-12B	0.40	1.60	0.51	0.20	1.60	2.20	1.50	1.90	4.29	4.30	4.00	4.90
Mistral-14B	0.00	0.00	0.00	0.00	0.30	0.30	0.60	0.50	0.00	2.60	0.10	0.10
Average	0.20	$0.80^{\uparrow 0.60}$	$0.25^{\uparrow 0.05}$	$0.10^{\downarrow 0.10}$	0.95	$1.25^{\uparrow 0.30}$	$1.05^{\uparrow 0.10}$	$1.20^{\uparrow 0.25}$	2.14	$3.45^{\uparrow 1.31}$	$2.05^{\downarrow 0.09}$	$2.50^{\uparrow 0.36}$
API Parameter Misuse Count												
Gemma-12B	1.51	3.80	1.41	1.82	0.30	0.30	0.30	0.00	2.86	12.40	2.30	3.27
Mistral-14B	0.71	1.00	0.60	0.61	0.00	0.10	0.40	0.20	0.41	2.70	0.00	0.41
Average	1.11	$2.40^{\uparrow 1.29}$	$1.01^{\downarrow 0.10}$	$1.21^{\uparrow 0.10}$	0.15	$0.20^{\uparrow 0.05}$	$0.35^{\uparrow 0.20}$	$0.10^{\downarrow 0.05}$	1.63	$7.55^{\uparrow 5.92}$	$1.15^{\downarrow 0.48}$	$1.84^{\uparrow 0.21}$

Static checking. As shown in Table 5, removing *Reasoning-Augmented Classification Retrieval* in *Variant 1* consistently lowers static correctness across all three libraries. Compared with DeepGroundRepair, *Variant 1* increases *API Hallucination Count* by 0.30–1.31 and *API Parameter Misuse Count* by 0.05–5.92 issues per 10 programs. By contrast, *Variant 2* and *Variant 3* have smaller and less consistent effects. Although they occasionally reduce issue counts for individual library–metric combinations, these improvements are isolated and do not generalize across libraries. Overall,

Table 6. *Code Executability Rate* (*CER*, %) for the complete DeepGroundRepair pipeline and its three variants, denoted in the table by DeepGR and V1–V3, respectively. "*Average*" denotes the arithmetic mean across the two models. Superscript values denote the relative change of each variant compared with DeepGR. The "↑" and "↓" indicate higher and lower *CER* (%), respectively.

Model	TensorFlow				PyTorch				PaddlePaddle			
	DeepGR	V1	V2	V3	DeepGR	V1	V2	V3	DeepGR	V1	V2	V3
Gemma-12B	59	52	57	44	34	38	21	17	21	5	13	12
Mistral-14B	59	55	62	49	54	54	49	36	48	31	43	31
Average	59.0	53.5$^{\downarrow 9.3\%}$	59.5$^{\uparrow 0.8\%}$	46.5$^{\downarrow 21.2\%}$	44.0	46.0$^{\uparrow 4.5\%}$	35.0$^{\downarrow 20.5\%}$	26.5$^{\downarrow 39.8\%}$	34.5	18.0$^{\downarrow 47.8\%}$	28.0$^{\downarrow 18.8\%}$	21.5$^{\downarrow 37.7\%}$

Table 7. *Semantic Consistency Rate* (*SCR*, %) for the complete DeepGroundRepair pipeline and its three variants, denoted in the table by DeepGR and V1–V3, respectively. "*Average*" denotes the arithmetic mean across the two models. Superscript values denote the relative change of each variant compared with DeepGR. The "↑" and "↓" indicate higher and lower *SCR*, respectively.

Model	TensorFlow				PyTorch				PaddlePaddle			
	DeepGR	V1	V2	V3	DeepGR	V1	V2	V3	DeepGR	V1	V2	V3
Gemma-12B	92	90	75	96	99	95	84	94	86	78	81	83
Mistral-14B	97	92	74	94	94	94	83	92	90	86	83	86
Average	94.5	91.0$^{\downarrow 3.7\%}$	74.5$^{\downarrow 21.2\%}$	95.0$^{\uparrow 0.5\%}$	96.5	94.5$^{\downarrow 2.1\%}$	83.5$^{\downarrow 13.5\%}$	93.0$^{\downarrow 3.6\%}$	88.0	82.0$^{\downarrow 6.8\%}$	82.0$^{\downarrow 6.8\%}$	84.5$^{\downarrow 4.0\%}$

the consistent performance degradation of *Variant 1* relative to DeepGroundRepair demonstrates the importance of *Reasoning-Augmented Classification Retrieval* in reducing API-level syntactic issues.

Dynamic checking. As shown in Table 6, *Variant 1* yields relative *CER* decreases of 9.3% on TensorFlow and 47.8% on PaddlePaddle, despite an increase of 4.5% on PyTorch. *Variant 2* exhibits a similar library-dependent pattern, decreasing *CER* by 20.5% on PyTorch and 18.8% on PaddlePaddle while producing a marginal increase of 0.8% on TensorFlow. Although their effects vary across libraries, removing either API knowledge grounding or root-cause localization generally reduces code executability. By contrast, *Variant 3* produces the largest and most consistent degradation, decreasing *CER* by 21.2%–39.8% across all three libraries. Its average relative decrease reaches 32.9%, substantially exceeding the decreases of 17.5% and 12.8% observed for *Variant 1* and *Variant 2*, respectively. Taken together, these results demonstrate that *Shape-Aware Self-Repair* makes the strongest contribution to improving code executability.

Semantic consistency. As shown in Table 7, *Variant 1* consistently reduces *SCR*, with relative decreases of 2.1%–6.8% across the three libraries. *Variant 3* has a limited and mixed effect, resulting in an average relative decrease of only 2.4%. By contrast, *Variant 2* produces the largest degradation, reducing *SCR* by 6.8%–21.2% across the three libraries, or 13.8% on average. Overall, the substantial performance degradation of *Variant 2* relative to DeepGroundRepair highlights the contribution of *Root-Cause Localization*, which directs repairs toward relevant faulty APIs and thereby helps preserve semantic consistency.

Summary of Answers to RQ2

Overall, the ablation results demonstrate that the three stages make complementary contributions to the performance of DeepGroundRepair. Stage 1 is the primary contributor to reducing API-level syntactic issues and also improves executability overall. Stage 2 makes the largest contribution to preserving semantic consistency while supporting executability by directing repairs toward localized root-cause APIs. Stage 3 makes the strongest and most consistent contribution to code executability while largely preserving semantic consistency.

5.3 RQ3: Stage-Specific Analysis

In this section, we evaluate how effectively the three stages of DeepGroundRepair address the corresponding challenges identified in Section 1: retrieving task-relevant API knowledge, localizing root-cause APIs, and repairing incorrect parameters under tensor-shape constraints. This analysis further clarifies the mechanisms underlying each stage's contribution and helps explain the ablation results in RQ2.

5.3.1 Stage 1: Reasoning-Augmented Classification Retrieval. We evaluate the retrieval accuracy of *Reasoning-Augmented Classification Retrieval* in DeepGroundRepair using *Precision@10* and *Recall@10*, with Hybrid-RAG as the baseline.

API ground-truth set construction. To establish relevance labels for evaluating retrieval accuracy, we construct a library-specific API ground-truth set for each task based on the reference implementation of DeepEval. Specifically, one author extracts API calls from the reference code, which is implemented in Keras, and maps them to corresponding or functionally equivalent APIs in TensorFlow, PyTorch, and PaddlePaddle. Another author then independently verifies all extracted APIs and mappings against the official library documentation to ensure their correctness and equivalence. The resulting task-specific sets are used as relevance labels for computing *Precision@10* and *Recall@10*.

Table 8. *Precision@10* (%) and *Recall@10* (%) for Hybrid-RAG in Baseline 2 and *Reasoning-Augmented Classification Retrieval* of DeepGroundRepair across models and DL libraries. Here, Baseline 2 and DeepGroundRepair are denoted by B2 and DeepGR.

Retrieval Method	Recommender LLM	TensorFlow		PyTorch		PaddlePaddle	
		Precision@10	*Recall@10*	*Precision@10*	*Recall@10*	*Precision@10*	*Recall@10*
Hybrid-RAG of B2	–	2.4	3.3	16.2	20.9	0.5	1.0
Stage 1 of DeepGR	Gemma-4B	62.0	84.8	59.7	77.7	42.8	62.1
	Gemma-12B	80.2	85.8	83.0	79.2	74.4	79.3
	Gemma-27B	89.9	89.5	79.9	79.2	73.8	77.5
	Mistral-3B	72.9	85.7	58.7	72.3	50.3	72.2
	Mistral-14B	73.6	94.4	65.6	79.9	59.5	73.8
	Mistral-24B	83.7	80.7	82.8	74.1	81.2	74.8

Overall result. As shown in Table 8, Stage 1 of DeepGroundRepair substantially outperforms Hybrid-RAG on both metrics across all 18 combinations of recommender LLMs and DL libraries. Averaged across the six LLMs, the improvements are 55.4–74.7 percentage points in *Precision@10* and 56.2–83.5 percentage points in *Recall@10* across the three libraries. Thus, Stage 1 consistently retrieves task-relevant APIs more accurately than Hybrid-RAG.

Table 9. An illustrative example of *Reasoning-Augmented Classification Retrieval* in PyTorch, showing how LLM-recommended APIs are verified and routed to sparse or dense retrieval based on their existence.

LLM-Recommended API	Existence	Retrieval Strategy	Retrieved API
`torch.nn.Conv2d`	✓	sparse retrieval	`torch.nn.Conv2d`
`torch.nn.MaxPool2d`	✓	sparse retrieval	`torch.nn.MaxPool2d`
`torch.nn.functional.add`	✘	dense retrieval	`torch.add`
`torch.nn.Linear`	✓	sparse retrieval	`torch.nn.Linear`

Mechanism analysis. *Reasoning-Augmented Classification Retrieval* verifies each LLM-recommended API and routes it according to its existence: existing APIs to sparse retrieval and hallucinated candidates to dense retrieval.

This strategy preserves the precise lexical signals of existing API names while exploiting the semantic information in hallucinated candidates to recover valid alternatives. As illustrated in Table 9, `torch.nn.functional.add` does not exist but captures the intended tensor-addition operation. *Reasoning-Augmented Classification Retrieval* therefore uses dense retrieval rather than sparse retrieval and recovers the valid API `torch.add`. By grounding draft code generation with more accurate API knowledge, this mechanism provides a stronger foundation for reducing syntactic issues.

5.3.2 Stage 2: Root-Cause Localization. Stage 2 aims to localize the root-cause API underlying an execution failure rather than merely the API at which the failure manifests. On programs that fail during initial execution, we use two complementary metrics to assess failure-site–root-cause discrepancies and the localization accuracy of Stage 2. *Failure-Site–Root-Cause Discrepancy Rate* (*FRDR*) measures the proportion of cases where the traceback-reported failure site is not the actual root-cause location. *Root-Cause API Localization Accuracy* (*RCLA*) measures the proportion of cases in which Stage 2 correctly identifies the manually annotated root-cause API.

Root-cause API ground-truth set construction. To balance representativeness and annotation cost, we used the medium-sized model from each family, Gemma-12B and Mistral-14B. Among the 351 programs that failed during initial execution, 312 exhibited tensor-shape mismatches. From these 312 cases, we selected 120 through stratified random sampling [34] by model and DL library. One author manually annotated the root-cause API of each failure based on the source code and traceback, without access to the predictions of Stage 2. A second author reviewed ambiguous cases to finalize the ground-truth labels. These labels were then used to evaluate the root-cause APIs localized by Stage 2.

Table 10. Evaluation of Stage 2 using *Failure-Site–Root-Cause Discrepancy Rate* (*FRDR*, %) and *Root-Cause API Localization Accuracy* (*RCLA*, %) on the sampled cases. For each library, "*Overall*" is calculated over all cases from both models.

Model	TensorFlow		PyTorch		PaddlePaddle	
	FRDR	*RCLA*	*FRDR*	*RCLA*	*FRDR*	*RCLA*
Gemma-12B	100.0	80.0	92.0	92.0	81.0	85.7
Mistral-14B	100.0	86.7	100.0	68.2	95.5	81.8
Overall	100.0	83.3	95.7	80.9	88.4	83.7

Overall result. As shown in Table 10, across all 120 cases, the overall *FRDR* is 94.2%, indicating that the reported failure site is generally a poor proxy for the underlying cause. Nevertheless, Stage 2 achieves an overall *RCLA* of 82.5%, with library-level accuracy ranging narrowly from 80.9% to 83.7%.

Mechanism analysis. The high *FRDR* indicates that a tensor incompatibility is usually reported downstream of the API that caused it, making traceback information alone insufficient for reliable root-cause localization. Stage 2 instead backtracks tensor-transformation dependencies from the reported failure site to upstream API configurations, enabling consistent localization accuracy across libraries. This effect is particularly evident for TensorFlow: despite an *FRDR* of 100.0%, Stage 2 achieves an *RCLA* of 83.3%.

5.3.3 Stage 3: Shape-Aware Self-Repair. To isolate the contribution of explicit tensor-shape guidance, we compare DeepGroundRepair with *Variant 3* on tensor-shape mismatch cases. Using the same model selection as in the Stage 2 analysis, we examine 312 such cases among 351 initially failing programs generated by Gemma-12B and Mistral-14B across all three DL libraries. Both methods use the same draft programs and Stage 2 localization results, with *Variant 3* omitting tensor-shape guidance.

Table 11. *Shape-Mismatch Error Repair Success Rate* (*ShapeRSR*, %) of DeepGroundRepair and Variant 3 on programs with initial tensor-shape mismatch failures. DeepGroundRepair and Variant 3 are denoted by DeepGR and V3, respectively. "Change" denotes the performance difference between V3 and DeepGR. The "↓" and "↑" indicate lower and higher *ShapeRSR*, respectively.

Model	Error Type	TensorFlow			PyTorch			PaddlePaddle		
		V3	DeepGR	Change	V3	DeepGR	Change	V3	DeepGR	Change
Gemma-12B	Intra-path	16.7	0.0	↓16.7	18.2	56.8	**↑38.6**	14.3	38.1	**↑23.8**
	Inter-path	6.3	56.3	**↑50.0**	0.0	0.0	–	0.0	0.0	–
Mistral-14B	Intra-path	0.0	25.0	**↑25.0**	14.9	53.2	**↑38.3**	9.3	51.2	**↑41.9**
	Inter-path	7.9	31.6	**↑23.7**	0.0	0.0	–	0.0	0.0	–

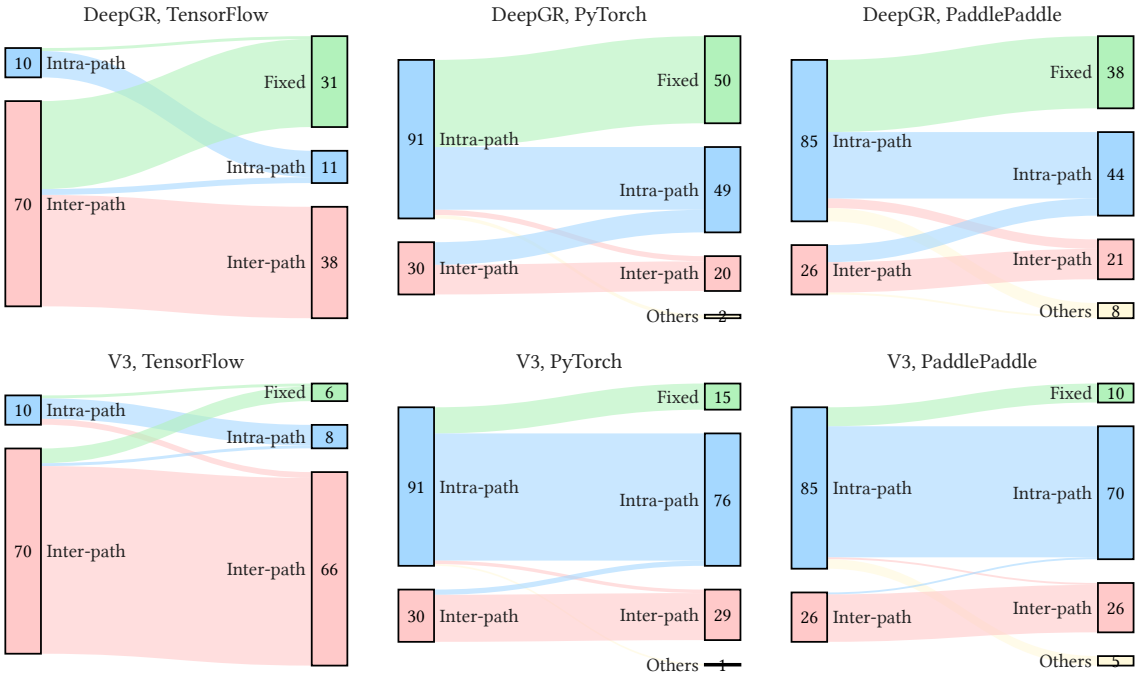


Fig. 7. Transition distributions of tensor shape mismatch failures from initial categories to final outcomes, aggregated across Gemma-12B and Mistral-14B. In the figure, DeepGroundRepair and Variant 3 are denoted by DeepGR and V3, respectively. "Fixed" indicates successful repair, while "Others" captures a small number of corner cases beyond the intra-path/inter-path taxonomy.

Overall result. Table 11 shows that explicit tensor-shape guidance generally improves *ShapeRSR* for programs with initial tensor-shape mismatch failures. Across the evaluated model–library configurations, DeepGroundRepair generally outperforms or matches *Variant 3*, with degradation occurring only occasionally. These results indicate that shape-aware repair effectively guides the LLM toward parameter changes that restore shape consistency. Nevertheless, some programs still fail after repair because resolving the initial shape mismatch may expose subsequent execution failures. We further analyze these remaining failures and their causes below.

Mechanism analysis. Figure 7 shows how programs with initial intra-path and inter-path mismatches transition to different execution outcomes after repair under *Variant 3* and DeepGroundRepair. Across both initial mismatch types,

the *Fixed* outcome contains 6 cases under *Variant 3* versus 31 under DeepGroundRepair in TensorFlow, 15 versus 50 in PyTorch, and 10 versus 38 in PaddlePaddle.

For initial intra-path mismatches, the improvement is particularly pronounced in PyTorch and PaddlePaddle. Among the 91 initial intra-path cases in PyTorch, the flow to *Fixed* contains 15 cases under *Variant 3*, compared with 50 under DeepGroundRepair. This difference suggests that explicit shape information helps the LLM produce parameter configurations that restore shape compatibility along a single execution path.

For initial inter-path mismatches, the outcome flows differ across libraries. In TensorFlow, more cases flow directly from *Inter-path* to *Fixed* under DeepGroundRepair. In PyTorch and PaddlePaddle, more cases instead flow from *Inter-path* to *Intra-path*. Specifically, among the 30 initial inter-path cases in PyTorch, this flow contains 3 cases under *Variant 3* versus 13 under DeepGroundRepair. Nevertheless, this transition indicates that the original inter-path incompatibility was resolved, allowing execution to expose a subsequent intra-path mismatch.

Summary of Answers to RQ3

Overall, the RQ3 results show that each stage contributes to addressing its targeted challenge. Stage 1 retrieves more task-relevant API knowledge, helping reduce API-level syntactic issues; Stage 2 localizes upstream root-cause APIs beyond traceback sites, narrowing the repair space; and Stage 3 repairs more tensor-shape mismatches, contributing to improved code executability. These stage-level findings complement the ablation results in RQ2 and help explain the end-to-end improvements observed in RQ1.

5.4 RQ4: Generalizability

To answer RQ4, we assess the generalizability of DeepGroundRepair by examining whether its effectiveness remains consistent across experimental configurations. Using the RQ1 results, we group the 18 model–library configurations along three dimensions: model family, model scale, and DL library. Figure 8 compares the group-level performance of DeepGroundRepair with the corresponding baseline averages.

5.4.1 Model Families. The "Model families" panels of Figure 8 present the results for the Gemma and Mistral families.

Comparison across model families. The two model families differ noticeably in their absolute performance. With DeepGroundRepair, Mistral produces 0.96 syntactic issues per 10 programs, compared with 4.58 for Gemma. Mistral also attains a higher *CER* of 48.8%, whereas Gemma reaches 38.4%. In contrast, *SCR* remains comparably high for both families, reaching 91.8% for Mistral and 94.9% for Gemma. These differences indicate that model family remains an important source of variation in the evaluated outcomes.

Consistency of improvements. DeepGroundRepair consistently outperforms the baseline averages for both families. It reduces syntactic issues from 12.22 to 4.58 for Gemma and from 4.13 to 0.96 for Mistral (top panel). It also increases *CER* from 20.2% to 38.4% and from 34.8% to 48.8% (middle panel), and *SCR* from 86.0% to 94.9% and from 74.1% to 91.8% (bottom panel). Thus, despite their different absolute performance, DeepGroundRepair provides consistent improvements across both model families.

5.4.2 Model Scales. For the following comparison, we denote the three parameter-scale ranges defined in Section 4.4.3 as small (below 10B), medium (10B–20B), and large (above 20B). The "Model scales" panels of Figure 8 present the results for these three scale groups.

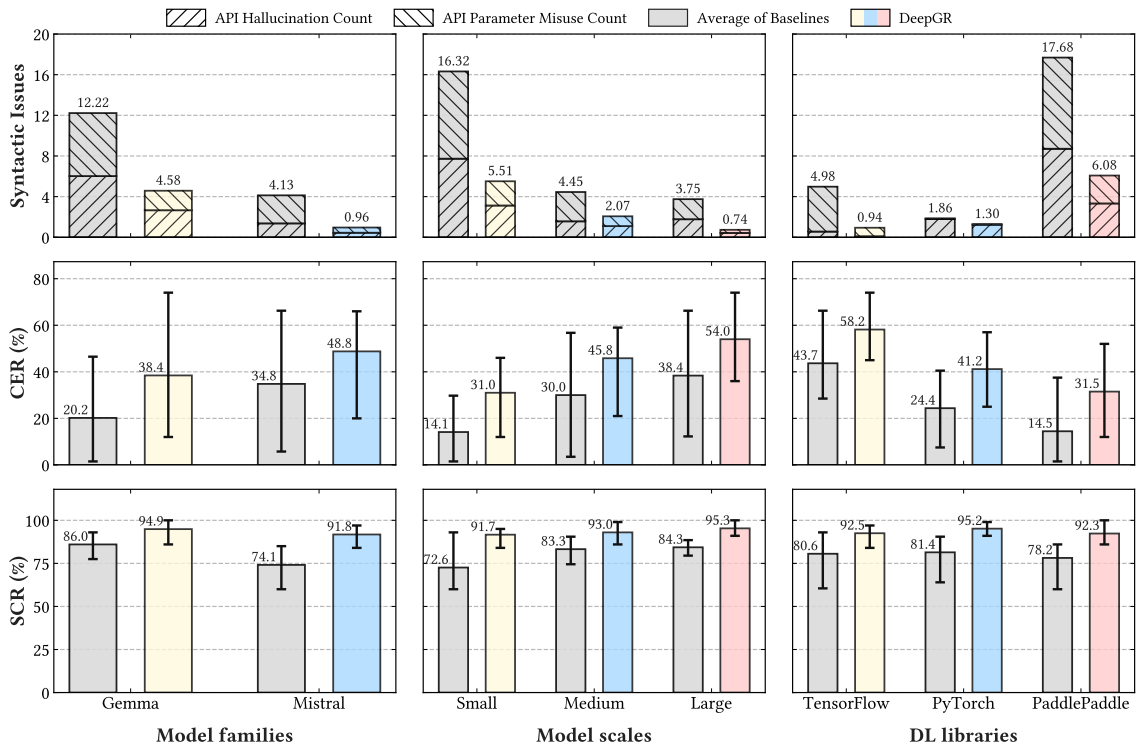


Fig. 8. Generalizability of DeepGroundRepair across model families, model scales, and DL libraries. Each bar pair compares the group-level averages of DeepGroundRepair with the corresponding baseline mean. Hatching in the syntactic-issue row distinguishes API hallucination counts from API parameter misuse counts. Whiskers in the *Code Executability Rate* (*CER*) and *Semantic Consistency Rate* (*SCR*) rows show the minimum and maximum configuration-level values within each group.

Comparison across model scales. Across these evaluated scale groups, models with more parameters exhibit fewer syntactic issues and higher *CER*. The small-model group produces 5.51 syntactic issues per 10 programs, whereas the medium- and large-model groups produce 2.07 and 0.74, respectively. Across the same groups, *CER* rises from 31.0% to 45.8% and then to 54.0%. By comparison, *SCR* varies only between 91.7% and 95.3%. These results suggest that model scale is more clearly associated with syntactic correctness and code executability.

Consistency of improvements. DeepGroundRepair consistently outperforms the corresponding baseline averages across all three scale groups. For the small-, medium-, and large-model groups, DeepGroundRepair achieves *CER* values of 31.0%, 45.8%, and 54.0%, respectively, compared with baseline averages of 14.1%, 30.0%, and 38.4%. Its corresponding *SCR* values are 91.7%, 93.0%, and 95.3%, exceeding the baseline averages of 72.6%, 83.3%, and 84.3%. Meanwhile, its syntactic-issue counts are 5.51, 2.07, and 0.74 per 10 programs, lower than the baseline averages of 16.32, 4.45, and 3.75. Taken together, these findings demonstrate consistent gains across all evaluated model scales.

5.4.3 DL Libraries. Finally, the "DL libraries" panels of Figure 8 present the results for the three target DL libraries: TensorFlow, PyTorch, and PaddlePaddle.

Comparison across DL libraries. The performance of DeepGroundRepair varies across the target DL libraries. On TensorFlow, it achieves the fewest syntactic issues, at 0.94 per 10 programs, and the highest *CER*, at 58.2%. PyTorch ranks second, with 1.30 issues and a *CER* of 41.2%, whereas PaddlePaddle is the most challenging, with 6.08 issues and a *CER* of 31.5%. By contrast, *SCR* remains stable across the three libraries, ranging from 92.3% to 95.2%.

Consistency of improvements. Despite variations in absolute performance, DeepGroundRepair consistently outperforms the corresponding baseline averages across all three libraries. On TensorFlow, PyTorch, and PaddlePaddle, it achieves *CER* values of 58.2%, 41.2%, and 31.5%, respectively, compared with baseline averages of 43.7%, 24.4%, and 14.5%. Its corresponding *SCR* values are 92.5%, 95.2%, and 92.3%, exceeding the baseline averages of 80.6%, 81.4%, and 78.2%. It also reduces the syntactic-issue counts to 0.94, 1.30, and 6.08 per 10 programs, compared with baseline averages of 4.98, 1.86, and 17.68. The particularly large reduction on PaddlePaddle indicates that DeepGroundRepair remains effective in the most challenging evaluated library setting.

Summary of Answers to RQ4

Overall, the RQ4 results support the generalizability of DeepGroundRepair across the evaluated settings. Across both model families, the three model-scale groups, and the three DL libraries, it consistently reduces API-level syntactic issues while improving code executability and semantic consistency relative to the baseline averages.

5.5 RQ5: Token Efficiency

To assess the token efficiency of DeepGroundRepair, we jointly analyze its code executability and token cost. We use code executability, measured by *CER*, as the effectiveness criterion because successful execution is a fundamental requirement for generated DL programs. Table 12 reports the detailed *Token Cost* across models, DL libraries, and methods, while Figure 9 visualizes the relationship between *Token Cost* and *CER*.

Table 12. Average *Token Cost* (in thousands of tokens) by model, DL library, and method. Here, DeepGR denotes DeepGroundRepair, and B1–B4 denote Baselines 1–4, respectively. "Δ (%)" reports the relative change of DeepGR over each corresponding baseline using the values in the corresponding "Overall Avg." row.

Model	TensorFlow					PyTorch					PaddlePaddle				
	B1	B2	B3	B4	DeepGR	B1	B2	B3	B4	DeepGR	B1	B2	B3	B4	DeepGR
Gemma-4B	0.53	0.54	1.09	1.10	1.10	0.61	0.61	1.23	1.23	**0.97**	0.72	0.57	1.44	1.12	**1.12**
Gemma-12B	0.75	0.70	1.56	1.43	**1.28**	0.76	0.73	1.55	1.50	**1.15**	0.76	0.70	1.54	1.41	**1.07**
Gemma-27B	0.65	0.65	1.36	1.34	**1.17**	0.76	0.75	1.56	1.54	**1.11**	0.77	0.74	1.56	1.48	**1.11**
Average	0.64	0.63	1.34	1.29	**1.18**	0.71	0.70	1.45	1.42	**1.08**	0.75	0.67	1.51	1.34	**1.10**
Mistral-3B	1.35	0.91	2.09	1.80	**1.50**	1.42	0.96	2.18	2.06	**1.33**	1.51	0.98	2.21	2.04	**1.58**
Mistral-14B	0.92	0.95	1.93	1.74	**1.17**	1.02	1.04	1.98	2.20	**1.35**	1.01	1.02	2.11	2.12	**1.17**
Mistral-24B	0.80	0.62	1.54	1.35	**0.89**	0.94	0.66	1.73	1.44	**0.96**	0.75	0.75	1.56	1.55	**0.85**
Average	1.02	0.83	1.85	1.63	**1.19**	1.13	0.89	1.96	1.90	**1.21**	1.09	0.92	1.96	1.90	**1.20**
Overall Avg.	0.83	0.73	1.60	1.46	**1.19**	0.92	0.79	1.71	1.66	**1.15**	0.92	0.79	1.74	1.62	**1.15**
Δ (%)	**↑43.4**	**↑63.0**	**↓25.6**	**↓18.5**	–	**↑25.0**	**↑45.6**	**↓32.7**	**↓30.7**	–	**↑25.0**	**↑45.6**	**↓33.9**	**↓29.0**	–

Compared with *Baselines 1 and 2*, DeepGroundRepair incurs 25.0%–63.0% higher *Token Cost* because it additionally repairs failed programs. This additional cost is accompanied by relative *CER* improvements of 54.0%–279.5%, as reported in RQ1. Because *Baselines 3 and 4* also involve both generation and repair, they provide a more comparable basis

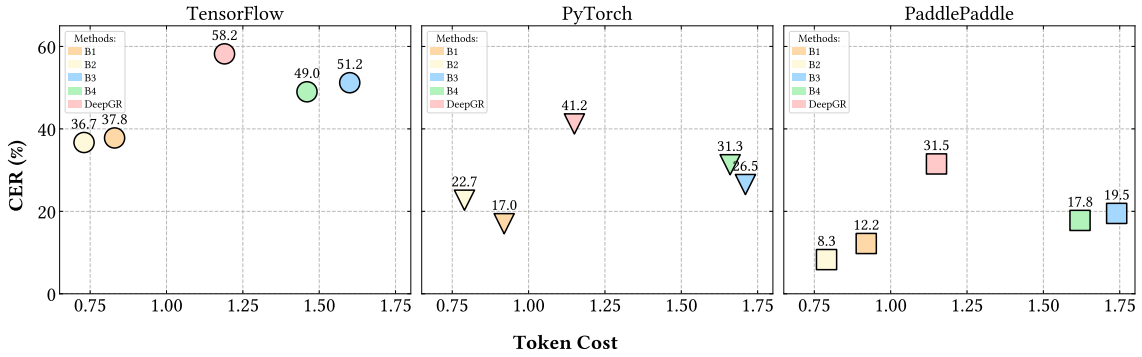


Fig. 9. Scatter plots comparing *Token Cost* (in thousands of tokens) and *Code Executability Rate* (*CER*, %) across methods and DL libraries. Each point represents the average *Token Cost* and *CER* of a method across the six evaluated models for a given DL library. DeepGR denotes DeepGroundRepair, while B1–B4 denote Baselines 1–4, respectively.

for evaluating token efficiency. Relative to these baselines, DeepGroundRepair reduces *Token Cost* by 18.5%–33.9% while improving *CER* by 13.7%–77.0%, thereby achieving both lower token cost and higher code executability than the self-debugging baselines.

As shown in Figure 9, the *Token Cost* and *CER* of each method are averaged across the six evaluated models and presented separately for the three DL libraries. Across all three DL libraries, DeepGroundRepair exhibits a consistent pattern: it achieves substantially higher *CER* than *Baselines 1 and 2* at an additional token cost, while outperforming *Baselines 3 and 4* in *CER* with lower token cost. Overall, these results demonstrate that DeepGroundRepair achieves a favorable trade-off between code executability and token cost across the evaluated settings.

Summary of Answers to RQ5

Overall, DeepGroundRepair demonstrates favorable efficiency in terms of code executability and token cost. Compared with the generation-only baselines, it incurs 25.0%–63.0% higher *Token Cost* but improves *CER* by 54.0%–279.5%. Relative to the self-debugging baselines, it improves *CER* by 13.7%–77.0% while reducing *Token Cost* by 18.5%–33.9%.

6 Discussion

In this section, we discuss the limitations and opportunities of our approach, followed by threats to validity.

6.1 Limitations of DeepGroundRepair

Although DeepGroundRepair is effective in improving LLM-generated DL programs, it has several limitations.

First, DeepGroundRepair primarily addresses API-level syntactic issues and tensor-shape mismatches, which are among the most prevalent failure categories observed in our evaluation. Its current design provides limited support for other DL-specific failures, including dtype or device incompatibilities, numerical instability, and training divergence. Incorporating failure-specific diagnostic capabilities and domain knowledge could enable more comprehensive analysis and repair of DL programs.

Second, the effectiveness of shape-aware analysis depends on observable tensor propagation and formalizable API semantics. Complex control flows, custom operators, and incomplete API documentation can hinder accurate tensor propagation tracing and shape constraint validation. Advanced program analysis techniques, such as symbolic execution, could strengthen tensor reasoning in these complex settings.

Finally, the current strategy performs parameter-level repair within a single iteration. Programs containing multiple independent errors may require repeated execution–repair cycles. Errors requiring architectural changes fall outside the scope of parameter-level repair and demand broader code revisions. Adaptive multi-step strategies could address these cases by dynamically determining the appropriate repair granularity and leveraging intermediate validation feedback. Such strategies could progressively resolve complex failures while maintaining repair efficiency.

6.2 Threats to Validity

We discuss potential threats to the construct, internal, and external validity of our study.

Construct Validity. The selected metrics may not fully capture the properties they are intended to measure. Pylint detects only predefined issue categories and may produce false positives, although we filter out most identifiable cases. Successful execution demonstrates runtime executability but does not guarantee functional correctness, while effective-line-count preservation only approximates semantic consistency. To reduce the influence of these limitations, we combine complementary static, dynamic, and semantic checks rather than relying on any single metric.

Internal Validity. The inherent stochasticity of LLMs may introduce variability in generated programs and affect method comparisons. To control its influence, we maintain consistent model configurations, prompts, and experimental environments within each comparison. In addition, constructing the ground-truth API sets and identifying the root-cause APIs for the stage-specific evaluations involve human judgment and may therefore introduce annotation bias. We mitigate this threat through consistent annotation guidelines and cross-checks against reference implementations, API documentation, and runtime traces.

External Validity. The generalizability of our findings may be limited by the scope of the evaluated tasks, LLMs, and DL libraries. We evaluate 100 model-level DL programming tasks from DeepEval, six open-weight LLMs from two model families, and three DL libraries. Although these settings cover diverse DL model architectures and library ecosystems, the results may not generalize to other LLMs, DL libraries, or large-scale real-world projects. Moreover, our evaluation focuses on API-related syntactic issues and tensor-shape mismatches, leaving other failure types, such as numerical instability and training-related issues, outside its scope. Broader evaluations across DL tasks, LLMs, libraries, real-world projects, and failure scenarios are therefore needed to assess generalizability beyond the studied settings.

7 Related Work

7.1 Bugs and Constraints in Deep Learning Programs

DL programs define model architectures by composing library-specific APIs into computation graphs, in which tensors are propagated and transformed across operations [1, 2]. Prior studies have consequently focused on two aspects: common bugs in DL programs and the constraints governing API usage and tensor transformations.

Bugs in Deep Learning Programs. Liang et al. [28] studied 64 real-world bugs in TensorFlow, PyTorch, and Keras programs and identified API misuse, tensor-shape mismatch, and type mismatch as common bug categories. Yang et al. [44] further found that DL API misuses differ from those of general-purpose APIs, frequently involving incorrect parameter types and missing API calls. More recently, Ma et al. [32] evaluated LLM-generated DL programs and reported

frequent API hallucinations and tensor-shape mismatches. The latter accounted for an average of 77.34% of execution failures. Together, these findings show that generating syntactically correct and executable DL programs requires correct API usage and consistent tensor shapes across operations.

Constraints in Deep Learning Programs. Beyond characterizing DL program bugs, recent studies have examined the ability of LLMs to extract and reason about API constraints. Xie et al. [43] evaluated LLMs for extracting formal specifications from comments and API documentation across TensorFlow, PyTorch, and MXNet. They considered data-type, structural, shape, and valid-value specifications. Their failure analysis identified *missing domain knowledge* as a major source of extraction errors. Focusing on numerical constraints in PyTorch and NumPy APIs, Deng et al. [13] further showed that LLM performance degrades when handling uncommon inputs and complex numerical relationships in APIs such as `Conv2d`, `reshape`, and `Fold`. This finding indicates that LLMs have limited robustness in reasoning about implicit numerical and shape constraints. Together, these observations motivate the use of explicit API knowledge and tensor-shape information when repairing DL programs.

7.2 Retrieval-Augmented Code Generation

Retrieval-augmented generation (RAG) improves code generation by incorporating external knowledge into the generation context [23, 41, 45]. Based on the retrieved knowledge source, existing approaches can be broadly divided into API documentation retrieval and relevant code retrieval.

API Documentation Retrieval. API documentation provides explicit knowledge of API functionality, signatures, parameters, and usage. Chen et al. [8] showed that documentation retrieval improves code generation, particularly for less commonly used libraries. Zhou et al. [47] proposed DocPrompting, which retrieves relevant documentation using sparse and dense retrieval. Jain et al. [21] showed that documentation augmentation can reduce low-frequency API hallucinations and proposed DAG++, which selectively retrieves documentation based on API existence and model confidence. These studies establish the value of API documentation for grounding code generation, but do not explicitly address the gap between high-level task requirements and the fine-grained APIs required for DL implementation. DeepGroundRepair addresses this gap by reasoning about task-relevant APIs before retrieval and routing candidates according to API existence.

Relevant Code Retrieval. Another line of work retrieves relevant code or repository context to provide implementation patterns. Gu et al. [17] showed that relevant file context and invoked-API information improve repository-level code generation, whereas irrelevant context can degrade performance. Pan et al. [36] combined sparse and dense code retrieval with statically extracted type context, while Li et al. [27] retrieved similar programs as in-context examples and generated intermediate artifacts to clarify requirements. These approaches primarily retrieve implementation context and reusable code patterns to guide generation. In contrast, DeepGroundRepair retrieves library-specific API specifications to ground API usage during generation and support tensor-shape reasoning during repair.

7.3 Repair of LLM-Generated Code

Existing approaches to repairing LLM-generated code can be broadly divided into training-based and feedback-guided methods. The former learn repair behavior through model optimization, whereas the latter use program feedback to guide repair at inference time.

Training-Based Code Repair. One line of work enhances code-repair capabilities through task-specific training on buggy programs and their corrected versions. Ding et al. [14] proposed CYCLE, which constructs refinement examples from model-generated faulty code, test feedback, and canonical solutions for iterative self-refinement. Jiang et al. [24] proposed LEDEX, which learns from explanation and refinement trajectories validated through execution, using supervised fine-tuning and reinforcement learning. Huang et al. [19] studied how code representations, fault-localization markers, and patch-generation strategies affect fine-tuned program repair. These approaches improve repair performance through dedicated training, whereas DeepGroundRepair performs repair at inference time without modifying model weights.

Feedback-Guided Code Repair. Another line of work uses behavioral information from generated programs to guide post-generation refinement. Chen et al. [11] proposed Self-Debugging, which prompts an LLM to iteratively inspect and revise its code using test results or self-generated explanations. Zhang et al. [46] proposed Self-Edit, which summarizes failures observed from example tests as fault-aware comments for a separately trained code editor to revise the program. Shi et al. [38] proposed MGDebugger, which structures repair according to program granularity by decomposing a program into hierarchical subfunctions, deriving local tests, and recursively repairing them from lower to higher levels. These methods demonstrate the effectiveness of program feedback for code repair. However, for DL programs, generic failure feedback provides limited guidance for identifying the upstream API configurations and tensor-shape dependencies underlying DL-specific failures. DeepGroundRepair instead combines root-cause localization with explicit tensor-shape guidance for parameter-level repair.

8 Conclusion and Future Work

This paper presents DeepGroundRepair, a three-stage framework for improving the syntactic correctness and runtime executability of LLM-generated DL programs. It targets API-level syntactic issues arising during code generation and tensor-shape mismatches that cause runtime failures. The framework progressively narrows the generation and repair spaces by retrieving task-relevant DL API knowledge, localizing the root-cause APIs of shape mismatches through static and dynamic analysis, and inferring shape-consistent API parameters from runtime tensor shapes.

We evaluate DeepGroundRepair on DeepEval using six open-weight LLMs across TensorFlow, PyTorch, and PaddlePaddle. Across the three libraries and four baselines, DeepGroundRepair substantially improves *CER*, achieving an average relative gain of 86.0%. It also reduces API hallucinations and API parameter misuses by 2.14 and 3.26 issues per 10 programs on average, respectively. Compared with the two self-debugging baselines, it improves *SCR* by an average of 16.7% across the three libraries. Together, these findings demonstrate the effectiveness of the generation–localization–repair design of DeepGroundRepair in reducing API-level syntactic issues, improving runtime executability, and preserving semantic consistency.

Future work will extend this generation–localization–repair paradigm to provide proactive and comprehensive support for DL code generation and repair. Integrating DL-specific constraints and lightweight verification into the generation process could help prevent failures before post-generation repair is required. Advanced program analyses could handle complex tensor flows. Adaptive multi-step repair strategies could address programs containing multiple interacting faults. Finally, we plan to expand diagnosis and repair beyond API-level issues and tensor-shape mismatches. Potential targets include dtype and device incompatibilities, numerical instability, and failures during training and deployment. Addressing these failures would provide broader support across the DL software lifecycle.

Acknowledgments

This work was supported by the National Natural Science Foundation of China under Grants 62372021, the National Key R&D Program of China under Grant 2024YFB3311503, the State Key Laboratory of Complex & Critical Software Environment under Grant SKLCCSE-2025ZX-02, and the Fundamental Research Funds for Beijing Municipal Universities under Grant 312000546325001.

References

[1] 2022. Tutorial on Directed Acyclic Graphs. *Journal of Clinical Epidemiology* 142 (Feb. 2022), 264–267. doi:10.1016/j.jclinepi.2021.08.001

[2] 2023. Exploring Keras Functional API for Complex Model Architectures - Ruta.Software. https://ruta.software/blog/exploring-keras-functional-api-for-complex-model-architectures.

[3] 2025. Gemma 3 Technical Report. arXiv:2503.19786 [cs.CL] https://arxiv.org/abs/2503.19786

[4] 2025. Mistralai/Mistral-Small-3.2-24B-instruct-2506 · Hugging Face. https://huggingface.co/mistralai/Mistral-Small-3.2-24B-Instruct-2506.

[5] 2026. Hugging Face – the AI Community Building the Future. https://huggingface.co.

[6] 2026. Pylint - Code Analysis for Python. https://www.pylint.org.

[7] 2026. Whoosh 2.7.4 Documentation — Whoosh 2.7.4 Documentation. https://whoosh.readthedocs.io/en/latest/.

[8] Jingyi Chen, Songqiang Chen, Jialun Cao, Jiasi Shen, and Shing-Chi Cheung. 2025. When LLMs Meet API Documentation: Can Retrieval Augmentation Aid Code Generation Just as It Helps Developers? arXiv:2503.15231 [cs] doi:10.48550/arXiv.2503.15231

[9] Jianlyu Chen, Shitao Xiao, Peitian Zhang, Kun Luo, Defu Lian, and Zheng Liu. 2024. M3-Embedding: Multi-Linguality, Multi-Functionality, Multi-Granularity Text Embeddings Through Self-Knowledge Distillation. In *Findings of the Association for Computational Linguistics: ACL 2024*, Lun-Wei Ku, Andre Martins, and Vivek Srikumar (Eds.). Association for Computational Linguistics, Bangkok, Thailand, 2318–2335. doi:10.18653/v1/2024.findings-acl.137

[10] Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Ponde de Oliveira Pinto, Jared Kaplan, Harri Edwards, Yuri Burda, Nicholas Joseph, Greg Brockman, and et al. 2021. Evaluating Large Language Models Trained on Code. arXiv:2107.03374 [cs]

[11] Xinyun Chen, Maxwell Lin, Nathanael Schärli, and Denny Zhou. 2023. Teaching Large Language Models to Self-Debug. arXiv:2304.05128 [cs.CL] doi:10.48550/arXiv.2304.05128

[12] Gordon V. Cormack, Charles L A Clarke, and Stefan Buettcher. 2009. Reciprocal Rank Fusion Outperforms Condorcet and Individual Rank Learning Methods *(SIGIR '09)*. Association for Computing Machinery, New York, NY, USA, 758–759. doi:10.1145/1571941.1572114

[13] Yinlin Deng, Chunqiu Steven Xia, Zhezhen Cao, Meiziniu Li, and Lingming Zhang. 2024. Can LLMs implicitly learn numeric parameter constraints in data science APIs?. In *Proceedings of the 38th International Conference on Neural Information Processing Systems* (Vancouver, BC, Canada) *(NIPS '24)*. Curran Associates Inc., Red Hook, NY, USA, Article 1718, 34 pages.

[14] Yangruibo Ding, Marcus J. Min, Gail Kaiser, and Baishakhi Ray. 2024. CYCLE: Learning to Self-Refine the Code Generation. *Proc. Acm Program. Lang.* 8, OOPSLA1 (April 2024), 108:392–108:418. doi:10.1145/3649825

[15] Matthijs Douze, Alexandr Guzhva, Chengqi Deng, Jeff Johnson, Gergely Szilvasy, Pierre-Emmanuel Mazaré, Maria Lomeli, Lucas Hosseini, and Hervé Jégou. 2026. The Faiss Library. *IEEE Transactions on Big Data* 12, 2 (2026), 346–361. doi:10.1109/TBDATA.2025.3618474

[16] Xueying Du, Geng Zheng, Kaixin Wang, Yi Zou, Yujia Wang, Wentai Deng, Jiayi Feng, Mingwei Liu, Bihuan Chen, Xin Peng, Tao Ma, and Yiling Lou. 2025. Vul-RAG: Enhancing LLM-based Vulnerability Detection via Knowledge-Level RAG. arXiv:2406.11147 [cs] doi:10.48550/arXiv.2406.11147

[17] Wenchao Gu, Juntao Chen, Yanlin Wang, Tianyue Jiang, Xingzhe Li, Mingwei Liu, Xilin Liu, Yuchi Ma, and Zibin Zheng. 2025. What to Retrieve for Effective Retrieval-Augmented Code Generation? An Empirical Study and Beyond. arXiv:2503.20589 [cs] doi:10.48550/arXiv.2503.20589

[18] Ari Holtzman, Jan Buys, Li Du, Maxwell Forbes, and Yejin Choi. 2019. The Curious Case of Neural Text Degeneration. In *International Conference on Learning Representations*.

[19] Kai Huang, Xiangxin Meng, Jian Zhang, Yang Liu, Wenjie Wang, Shuhao Li, and Yuqing Zhang. 2023. An Empirical Study on Fine-Tuning Large Language Models of Code for Automated Program Repair. In *2023 38th IEEE/ACM International Conference on Automated Software Engineering (ASE)*. 1162–1174. doi:10.1109/ASE56229.2023.00181

[20] Nargiz Humbatova, Gunel Jahangirova, Gabriele Bavota, Vincenzo Riccio, Andrea Stocco, and Paolo Tonella. 2020. Taxonomy of Real Faults in Deep Learning Systems. In *ICSE '20: 42nd International Conference on Software Engineering (ICSE '20)*. Association for Computing Machinery, New York, NY, USA, 1110–1121. doi:10.1145/3377811.3380395

[21] Nihal Jain, Robert Kwiatkowski, Baishakhi Ray, Murali Krishna Ramanathan, and Varun Kumar. 2025. On Mitigating Code LLM Hallucinations with API Documentation. In *2025 IEEE/ACM 47th International Conference on Software Engineering: Software Engineering in Practice (ICSE-SEIP)*. 237–248. doi:10.1109/ICSE-SEIP66354.2025.00027

[22] Juyong Jiang, Fan Wang, Jiasi Shen, Sungju Kim, and Sunghun Kim. 2026. A Survey on Large Language Models for Code Generation. *ACM Trans. Softw. Eng. Methodol.* 35, 2, Article 58 (Jan. 2026), 72 pages. doi:10.1145/3747588

[23] Juyong Jiang, Fan Wang, Jiasi Shen, Sungju Kim, and Sunghun Kim. 2026. A survey on large language models for code generation. *ACM Transactions on Software Engineering and Methodology* 35, 2 (2026), 1–72.

[24] Nan Jiang, Xiaopeng Li, Shiqi Wang, Qiang Zhou, Soneya B Hossain, Baishakhi Ray, Varun Kumar, Xiaofei Ma, and Anoop Deoras. 2024. Ledex: Training LLMs to better self-debug and explain code. *Advances in Neural Information Processing Systems* 37 (2024), 35517–35543.

[25] Xue Jiang, Yihong Dong, Yongding Tao, Huanyu Liu, Zhi Jin, and Ge Li. 2025. ROCODE: Integrating Backtracking Mechanism and Program Analysis in Large Language Models for Code Generation. In *2025 IEEE/ACM 47th International Conference on Software Engineering (ICSE)*. 334–346. doi:10.1109/ICSE55347.2025.00133

[26] Jia Li, Ge Li, Yongmin Li, and Zhi Jin. 2025. Structured Chain-of-Thought Prompting for Code Generation. *ACM Trans. Softw. Eng. Methodol.* 34, 2, Article 37 (Jan. 2025), 23 pages. doi:10.1145/3690635

[27] Jia Li, Yunfei Zhao, Yongmin Li, Ge Li, and Zhi Jin. 2024. AceCoder : An Effective Prompting Technique Specialized in Code Generation. *Acm Transactions On Software Engineering and Methodology* 33, 8 (Nov. 2024), 1–26. doi:10.1145/3675395

[28] Yunkai Liang, Yun Lin, Xuezhi Song, Jun Sun, Zhiyong Feng, and Jin Song Dong. 2022. gDefects4DL: A Dataset of General Real-World Deep Learning Program Defects. In *Proceedings of the ACM/IEEE 44th International Conference on Software Engineering: Companion Proceedings*. ACM, Pittsburgh Pennsylvania, 90–94. doi:10.1145/3510454.3516826

[29] Alexander H. Liu, Kartik Khandelwal, Sandeep Subramanian, Victor Jouault, Abhinav Rastogi, Adrien Sadé, Alan Jeffares, Albert Jiang, Alexandre Cahill, Alexandre Gavaudan, Alexandre Sablayrolles, et al. 2026. Ministral 3. arXiv:2601.08584 [cs.CL] doi:10.48550/arXiv.2601.08584

[30] Nelson F Liu, Kevin Lin, John Hewitt, Ashwin Paranjape, Michele Bevilacqua, Fabio Petroni, and Percy Liang. 2024. Lost in the middle: How language models use long contexts. *Transactions of the association for computational linguistics* 12 (2024), 157–173.

[31] Yi Luan, Jacob Eisenstein, Kristina Toutanova, and Michael Collins. 2021. Sparse, dense, and attentional representations for text retrieval. *Transactions of the Association for Computational Linguistics* 9 (2021), 329–345.

[32] Xiangyue Ma, Xiaoting Du, Chenglong Li, Jiangtao Meng, Xiaoke Fang, Wenjie Ding, and Zheng Zheng. 2026. How Do Large Language Models Perform in Deep Learning Code Generation? An Empirical Study. *ACM Trans. Softw. Eng. Methodol.* (May 2026). Just Accepted. doi:10.1145/3816024

[33] Xiangyue Ma, Zhen Wang, Xuhui Lu, Xiaoting Du, Chenglong Li, Jun Li, Haotian Li, and Zheng Zheng. 2026. Artifacts for "DeepGroundRepair: From API-Grounded Generation to Shape-Aware Repair for Deep Learning Programs". Zenodo. https://doi.org/10.5281/zenodo.22727085.

[34] Kimberly A Neuendorf. 2017. *The content analysis guidebook*. sage.

[35] Theo X Olausson, Jeevana Priya Inala, Chenglong Wang, Jianfeng Gao, and Armando Solar-Lezama. 2024. Is self-repair a silver bullet for code generation?. In *International Conference on Learning Representations*, Vol. 2024. 36545–36593.

[36] Zhiyuan Pan, Xing Hu, Xin Xia, and Xiaohu Yang. 2025. CATCODER: Repository-level Code Generation with Relevant Code and Type Context. *Acm Trans. Softw. Eng. Methodol.* (Dec. 2025), 1–24. doi:10.1145/3779217

[37] Stephen Robertson and Hugo Zaragoza. 2009. The probabilistic relevance framework: BM25 and beyond. *Foundations and trends® in information retrieval* 4, 1-2 (2009), 1–174.

[38] Yuling Shi, Songsong Wang, Chengcheng Wan, Min Wang, and Xiaodong Gu. 2024. From Code to Correctness: Closing the Last Mile of Code Generation with Hierarchical Debugging. doi:10.48550/ARXIV.2410.01215

[39] Jiho Shin, Moshi Wei, Junjie Wang, Lin Shi, and Song Wang. 2024. The Good, the Bad, and the Missing: Neural Code Generation for Machine Learning Tasks. *ACM Transactions on Software Engineering and Methodology* 33, 2 (Feb. 2024), 1–24. arXiv:2305.09082 [cs] doi:10.1145/3630009

[40] Hongjin Su, Shuyang Jiang, Yuhang Lai, Haoyuan Wu, Boao Shi, Che Liu, Qian Liu, and Tao Yu. 2024. EvoR: Evolving Retrieval for Code Generation. In *Findings of the Association for Computational Linguistics: EMNLP 2024*, Yaser Al-Onaizan, Mohit Bansal, and Yun-Nung Chen (Eds.). Association for Computational Linguistics, Miami, Florida, USA, 2538–2554. doi:10.18653/v1/2024.findings-emnlp.143

[41] Zora Zhiruo Wang, Akari Asai, Xinyan Velocity Yu, Frank F. Xu, Yiqing Xie, Graham Neubig, and Daniel Fried. 2025. CodeRAG-Bench: Can Retrieval Augment Code Generation?. In *Findings of the Association for Computational Linguistics: NAACL 2025*, Luis Chiruzzo, Alan Ritter, and Lu Wang (Eds.). Association for Computational Linguistics, Albuquerque, New Mexico, 3199–3214. doi:10.18653/v1/2025.findings-naacl.176

[42] Moshi Wei, Nima Shiri Harzevili, Yuchao Huang, Yang Jinqiu, Junjie Wang, and Song Wang. 2022. Demystifying and Detecting Misuses of Deep Learning APIs. In *Proceedings of the 46th International Conference on Software Engineering*. ACM.

[43] Danning Xie, Byoungwoo Yoo, Nan Jiang, Mijung Kim, Lin Tan, Xiangyu Zhang, and Judy S Lee. 2025. How effective are large language models in generating software specifications?. In *2025 IEEE International Conference on Software Analysis, Evolution and Reengineering (SANER)*. IEEE, 1–12.

[44] Deheng Yang, Kui Liu, Yan Lei, Li Li, Huan Xie, Chunyan Liu, Zhenyu Wang, Xiaoguang Mao, and Tegawendé F. Bissyandé. 2024. Demystifying API Misuses in Deep Learning Applications. *Empirical Software Engineering* 29, 2 (Feb. 2024), 45. doi:10.1007/s10664-023-10413-9

[45] Zezhou Yang, Sirong Chen, Cuiyun Gao, Zhenhao Li, Xing Hu, Kui Liu, and Xin Xia. 2025. An empirical study of retrieval-augmented code generation: Challenges and opportunities. *ACM Transactions on Software Engineering and Methodology* 34, 7 (2025), 1–28.

[46] Kechi Zhang, Zhuo Li, Jia Li, Ge Li, and Zhi Jin. 2023. Self-Edit: Fault-aware Code Editor for Code Generation. In *ACL 2023*, Anna Rogers, Jordan Boyd-Graber, and Naoaki Okazaki (Eds.). Association for Computational Linguistics, Toronto, Canada, 769–787. doi:10.18653/v1/2023.acl-long.45

[47] Shuyan Zhou, Uri Alon, Frank F. Xu, Zhiruo Wang, Zhengbao Jiang, and Graham Neubig. 2023. DocPrompting: Generating Code by Retrieving the Docs. arXiv:2207.05987 [cs] doi:10.48550/arXiv.2207.05987

A Prompting Templates

This appendix presents the prompt templates used by DeepGroundRepair. To ensure a fair comparison, prompt components unrelated to method-specific designs are kept identical across DeepGroundRepair and the baselines. The complete prompts for all methods are available in our open-source repository [33].

A.1 Stage 1: *Reasoning-Augmented Classification Retrieval*

As described in Section 3.1, Stage 1 involves two LLM interactions for API recommendation and code generation. The former recommends candidate API names from the task requirement for subsequent verification and retrieval, whereas the latter generates a DL program grounded in the retrieved API documentation.

A.1.1 API Recommendation. As shown in Figure 10, the prompt combines the DeepEval task requirement, recommendation instructions, and an output format to elicit a structured list of candidate API names.

There is a deep learning code generation task described below. Please provide only the most relevant {*DL Library*} APIs for constructing the model.

[Task Requirement]:
{*Task Requirement*}

[Task Instructions]:
Return only valid Python code in the following exact format: a list of dictionaries, each containing the selected API name.

[Output Format]:
candidate_apis = [{"api_name": ...}, ...]

Fig. 10. Prompting template of API recommendation in stage 1.

A.1.2 Draft Code Generation. As shown in Figure 11, the prompt combines the task requirement, generation instructions, an output format, and retrieved documentation containing the names, signatures, and descriptions of relevant APIs. These components guide the LLM in generating a complete model-level DL program.

A.2 Stage 3: *Shape-Aware Self-Repair*

Stage 3 uses separate prompt templates for the intra-path and inter-path repair strategies described in Section 3.3. The former provides local tensor-shape context, whereas the latter additionally supplies shape-related API knowledge for satisfying cross-path constraints.

A.2.1 Intra-path Mismatch Repair. As shown in Figure 12, the prompt includes shape-annotated and parameter-masked code, the localized buggy API call, repair instructions, and an output format. The shape annotations provide the local tensor-shape constraints needed to infer the masked parameter values, while the repair instructions restrict changes to the localized buggy API call.

A.2.2 Inter-path Mismatch Repair. As shown in Figure 13, the prompt combines shape-annotated and parameter-masked code, cross-path shape constraints, retrieved API documentation, repair instructions, and an output format. The documentation describes the masked parameters and provides the tensor-shape transformation formula of the buggy API, guiding the LLM to infer parameter values that satisfy the cross-path constraints.

You are a developer specializing in deep learning. Complete the deep learning code generation task described below with reference to the provided API documentation.

[Task Requirement]:
{*Task Requirement*}

[API Document]:
API Name: {*API Name*}
API Description: {*API Description*}
API Signature: {*API Signature*}

[Task Instructions]:
(1) Import {*DL Library*} and all necessary APIs from {*DL Library*} referenced in the API document;
(2) Complete the DL_Model class by inheriting from the appropriate base model class provided by {*DL Library*} ;
(3) Specify all layer API parameters using explicit keyword arguments;
(4) Instantiate the DL_Model class and construct a dummy input tensor according to the expected input shape to validate the generated model.

[Output Format]:

```
class DL_Model:
    # your code here

if __name__ == "__main__":
    model = DL_Model()
    input_tensor = ...
    output_tensor = model(input_tensor)
```

Fig. 11. Prompting template of draft code generation in stage 1.

You are a deep learning code repair assistant. Given tensor-shape-annotated code and a buggy API call in which suspicious parameter values are masked, repair the buggy API call.

[Tensor-Shape-Annotated and Buggy-Parameter-Masked Code]:
{*Tensor Shape Annotated Code*}

[Buggy API Call with Masked Parameters]:
{*Masked Buggy API Call*}

[Task Instructions]:
(1) Examine the masked API call and identify all parameters whose values are replaced with '*mask*'.
(2) Infer valid values for the masked parameters based on the annotated tensor shapes.
(3) Replace only the '*mask*' placeholders with valid parameter values, ensuring that the repaired API call satisfies all tensor-shape constraints in the given context.
(4) Preserve the original structure of the masked API call.
(5) Return only the repaired API call as a single line of valid Python code, without explanations or Markdown code fences.

[Output Format]:

```
repaired_api_call(...)
```

Fig. 12. Prompting template of intra-path mismatch repair in stage 3.

You are a deep learning tensor-shape constraint-solving assistant. Your task is to infer valid values for the masked parameters. When substituted into the tensor-shape transformation formula, these values should enable the masked API call to transform its input tensor shape exactly into the target output tensor shape.

[Tensor-Shape-Annotated and Buggy-Parameter-Masked Code]:
{*Tensor Shape Annotated Code*}

[Extracted Buggy API Call]:

[Shape Constraint-Solving Information]:
Input Tensor Shape: {*Input Tensor Shape*}
Masked API Call: {*Masked API Call*}
Target Output Tensor Shape: {*Target Output Tensor Shape*}
Tensor-Shape Transformation Formula: {*Tensor-Shape Transformation Formula*}

[API Documentation]:
API Name: {*API Name*}
API Parameters: {*API Parameters*}

[Task Instructions]:
(1) Identify the masked parameters.
- Examine each masked API call and identify only the parameters whose values are masked.

(2) Extract the relevant constraint information.
- Examine the tensor-shape-annotated code, shape constraint-solving information, and API documentation. Extract the input tensor shape, target output tensor shape, tensor-shape transformation formula, and semantics of the masked parameters.

(3) Generate and verify candidate parameter combinations.
- Generate up to five candidate value combinations for the masked parameters.
- For each candidate combination, substitute its values into the tensor-shape transformation formula and compute the resulting output tensor shape.
- Retain only candidate combinations whose computed output tensor shape exactly matches the target output tensor shape.

(4) Return the reasoning process and final answer.
- Complete the preceding tasks step by step. First provide the reasoning process, and then return the final answer as a JSON object conforming exactly to the schema below.

[Output Format]:
```
{
  "masked_api_call": "<masked API call>",
  "candidate_params_combinations": [
    {"<masked_param_1>": <value>},
    {"<masked_param_2>": <value>}
  ]
}
```

Fig. 13. Prompting template of inter-path mismatch repair in stage 3.